JS

ECMASCRIPT 2024 / 2025 · ES15 / ES16

The

JavaScript Reference

A complete reference to the JavaScript language (ECMAScript), from lexical structure to metaprogramming.

FOUNDATIONS · FUNCTIONS AND OBJECTS · DATA STRUCTURES · STANDARD LIBRARY · ERRORS, ASYNC, MODULES · ADVANCED AND ENVIRONMENTAL

How to Use This Book

This is a complete reference to the JavaScript language, organised in six parts that move from the lexical rules of the language through functions and objects, data structures, the standard library, asynchronous programming and modules, and finally the host environments and tooling around it.

> Version basis: ECMAScript 2024/2025 (ES15/ES16), with notes on where features landed.

Reading it

Each part opens with a list of its chapters. Each chapter opens with a short *In this chapter* list so you can jump straight to the section you need. Read Part I and II in order if you are building foundations; dip into Parts III–VI as a reference.

Looking things up

- **Contents** — every part and chapter, with page numbers.
- **Cross-references** — wherever the text says something like §18.2 p. 70, the reference is a live link and is followed by the page it points to.
- **Section Index** — at the back, every section title in alphabetical order, so you can find "Temporal Dead Zone" or "Event Delegation" without knowing which chapter it lives in.
- **PDF bookmarks** — the full part → chapter → section outline is available in your PDF reader's sidebar.

Conventions

Code is set in a monospaced face with syntax highlighting. Comments in examples show results (`// → 3`) or call out a pitfall (`// ← TypeError`). Running heads show the current chapter on the left page and the current section on the right page.

Contents

PART I **Foundations** 1

1 Lexical Structure ... 2

2 Types and Values ... 6

3 Variables, Scope, and Hoisting ... 11

4 Type Coercion and Conversion ... 15

5 Operators and Expressions ... 20

6 Statements and Control Flow ... 25

PART II **Functions and Objects** 29

7 Functions ... 30

8 Closures ... 36

9 this and Execution Context ... 38

10 Objects ... 41

11 Property Descriptors and Object Meta-Operations ... 46

12 Prototypes and Inheritance ... 48

13 Classes ... 51

PART III **Data Structures** 55

14 Arrays ... 56

15 Destructuring, Spread, and Rest ... 61

16 Iterators and Iterables ... 64

17 Generators ... 66

PART IV **Standard Library** 69

18 String ... 70

19 Number and Math ... 74

20 BigInt ... 77

21 Date and Temporal ... 78
22 Regular Expressions ... 81
23 JSON ... 85
24 Map, Set, WeakMap, WeakSet, WeakRef ... 87
25 Symbol ... 90
26 Proxy and Reflect ... 92
27 Typed Arrays, ArrayBuffer, DataView ... 94
28 Intl — Internationalization ... 97

PART V **Errors, Async, Modules** 99

29 Errors and Exception Handling ... 100
30 The Event Loop and Concurrency Model ... 103
31 Promises ... 106
32 async / await ... 110
33 Async Iteration ... 113
34 Modules ... 115

PART VI **Advanced and Environmental** 119

35 Strict Mode ... 120
36 Memory Management and Garbage Collection ... 122
37 Metaprogramming Recap ... 124
38 Feature History by ECMAScript Version ... 125
39 Gotchas, Pitfalls, and Quirks ... 128
40 The Browser Environment ... 131
41 The Node.js Environment ... 134
42 Tooling Ecosystem ... 136
43 Appendices ... 138

Section Index 141

PART I

Foundations

1	Lexical Structure	2
2	Types and Values	6
3	Variables, Scope, and Hoisting	11
4	Type Coercion and Conversion	15
5	Operators and Expressions	20
6	Statements and Control Flow	25

CHAPTER 1

Lexical Structure

IN THIS CHAPTER

1.1 Character Set and Encoding 2
1.2 Case Sensitivity 2
1.3 Whitespace and Line Terminators 2
1.4 Comments 3
1.5 Identifiers 3
1.6 Reserved Words 4
1.7 Literals 4
1.8 Semicolons and Automatic Semicolon Insertion (ASI) 4
1.9 Grammar Notes 5

The lexical structure is the set of rules for writing the characters of a program: what counts as a token, how whitespace works, how comments work.

1.1 Character Set and Encoding

JavaScript source is Unicode. Internally, strings and identifiers are sequences of **UTF-16 code units**. This has consequences everywhere (see §18.2 p. 70).

```
const café = 1;      // non-ASCII identifiers are legal
const 变量  = 2;
const \u0061bc = 3;  // Unicode escapes in identifiers → same as `abc`
```

1.2 Case Sensitivity

JavaScript is case-sensitive. `myVar`, `myvar`, and `MyVar` are three different identifiers. Keywords are lowercase (`if`, `while`, `class`); built-in constructors are capitalized (`Object`, `Array`, `Promise`).

1.3 Whitespace and Line Terminators

Whitespace characters recognized between tokens:

Code point	Name
`U+0009`	Tab
`U+000B`	Vertical tab

Code point	Name
`U+000C`	Form feed
`U+0020`	Space
`U+00A0`	No-break space
`U+FEFF`	Zero-width no-break space (BOM)
Unicode `Zs` category	Other space separators

Line terminators (significant for ASI and for `//` comments):

Code point	Name
`U+000A`	Line feed `\n`
`U+000D`	Carriage return `\r`
`U+2028`	Line separator
`U+2029`	Paragraph separator

1.4 Comments

```
// single-line comment, runs to end of line

/* multi-line
   comment */

/** JSDoc-style — just a block comment, but tooling reads it */

#!/usr/bin/env node   // hashbang, only legal as the very first line (ES2023)
```

Block comments do not nest. `/* /* */ */` is a syntax error at the trailing `*/`.

1.5 Identifiers

An identifier must start with a letter, `$`, `_`, or a Unicode escape sequence, and may continue with those plus digits and certain Unicode combining marks. Formally: starts with `ID_Start` ∪ `{$, _}`, continues with `ID_Continue` ∪ `{$, _, ZWNJ, ZWJ}`.

```
let _private, $jquery, camelCase, PascalCase, CONSTANT_CASE;
let π = Math.PI;
// let 1abc;   ← SyntaxError: cannot start with a digit
```

1.6 Reserved Words

Always reserved:

```
await* break case catch class const continue debugger default delete do
else enum export extends false finally for function if import in
instanceof new null return super switch this throw true try typeof
var void while with yield*
```

`await` is reserved inside modules and async functions; `yield` inside generators and strict mode.

Reserved in strict mode only: `implements`, `interface`, `let`, `package`, `private`, `protected`, `public`, `static`.

Contextual keywords (legal as identifiers, but special in certain positions): `as`, `async`, `from`, `get`, `set`, `of`, `target`, `meta`, `arguments`, `eval` (last two are restricted in strict mode).

Future reserved / historically reserved: `enum` is reserved and unused. `abstract`, `boolean`, `byte`, `char`, `double`, `final`, `float`, `goto`, `int`, `long`, `native`, `short`, `synchronized`, `throws`, `transient`, `volatile` were reserved in ES3 and are now free.

1.7 Literals

```
42                // number
3.14              // number
0b1010            // binary → 10
0o755             // octal → 493
0xFF              // hex → 255
1_000_000         // numeric separator (ES2021)
1e3               // 1000
.5                // 0.5
5.                // 5
9007199254740993n // BigInt
'single'          // string
"double"          // string
`template ${x}`   // template literal
/ab+c/gi          // regex
true, false, null // keyword literals
[1, 2, 3]         // array literal
{ a: 1 }          // object literal
```

1.8 Semicolons and Automatic Semicolon Insertion (ASI)

Semicolons terminate statements. The parser will insert them for you under three rules:

1. When a token cannot be parsed as a continuation of the current statement and there is a line break before it, a semicolon is inserted.

2. When the end of input is reached and the program cannot be parsed, a semicolon is inserted.
3. When a *restricted production* is violated — a line break appears where the grammar forbids one.

Restricted productions (a newline here forces a semicolon):

```
return \n expr        // → return; expr;
throw  \n expr        // → SyntaxError (throw requires an operand)
break  \n label       // → break;
continue \n label     // → continue;
expr \n ++            // postfix ++/-- must be on the same line
async \n function     // → not an async function
yield \n expr         // in generators
```

The classic bug:

```
function f() {
  return
    { ok: true };
}
f(); // undefined — ASI inserted a semicolon after `return`
```

Lines that are dangerous to start (they continue the previous line): `(`, `[`, `` ` ``, `+`, `-`, `/`, `,`, `.`. Defensive style prefixes them with a semicolon:

```
const a = b
;[1, 2].forEach(fn)   // without the leading `;`, this parses as b[1,2]
```

1.9 Grammar Notes

- **Expression vs. Statement position** matters. A `{` at statement start is a block, not an object literal. `function` at statement start is a declaration, not an expression.
- Wrapping in parentheses forces expression context: `({}).toString()`, `(function(){})()`.
- **Directive prologue:** string literal expression statements at the top of a script/function body, e.g. `"use strict"`.

CHAPTER 2

Types and Values

IN THIS CHAPTER

2.1 The Type System at a Glance 6
2.2 Undefined 7
2.3 Null 7
2.4 Boolean 7
2.5 Number 8
2.6 String 8
2.7 Symbol 8
2.8 BigInt 9
2.9 Object 9
2.10 Wrapper Objects (Boxing) 9
2.11 Detecting Types Reliably 10

2.1 The Type System at a Glance

JavaScript is **dynamically typed** (types attach to values, not variables) and **weakly typed** (implicit coercion is pervasive).

There are **8 types:** 7 primitives + object.

Type	`typeof`	Notes
Undefined	`"undefined"`	Single value `undefined`
Null	`"object"` ⚠	Single value `null`; the `typeof` result is a famous bug
Boolean	`"boolean"`	`true` / `false`
Number	`"number"`	IEEE-754 double
BigInt	`"bigint"`	Arbitrary-precision integer (ES2020)
String	`"string"`	UTF-16 code unit sequence
Symbol	`"symbol"`	Unique property keys (ES2015)
Object	`"object"` or `"function"`	Everything else

Primitives are **immutable** and compared **by value.** Objects are **mutable** and compared **by reference.**

2.2 Undefined

`undefined` is the value of:

- a declared but unassigned variable
- a missing function parameter
- a missing object property
- a function with no `return` (or bare `return`)
- the result of `void anything`

```
let x;                  // undefined
function f(a) { return a; }
f();                    // undefined
({}).nope;              // undefined
[1,2,3][99];            // undefined
void 0;                 // undefined — the canonical safe way to write it
```

`undefined` is a *property of the global object*, writable in sloppy pre-ES5 environments but non-writable since ES5. It is **not** a reserved word, so it can be shadowed locally:

```
(function (undefined) { /* `undefined` is guaranteed correct here */ })();
```

2.3 Null

`null` is an intentional absence of a value — an "empty pointer." It is a keyword literal, not a global property, and can never be shadowed.

```
typeof null;                // "object"  ← historical bug, cannot be fixed
null === undefined;         // false
null == undefined;          // true
Object.getPrototypeOf({});  // Object.prototype
Object.create(null);        // object with NO prototype ("dict object")
```

Convention: `undefined` = "not set / doesn't exist"; `null` = "deliberately set to nothing."

2.4 Boolean

Only `true` and `false`. See §4.2 p. 15 for what converts to which.

```
Boolean(x)   // explicit conversion
!!x          // idiomatic explicit conversion
```

2.5 Number

All JS numbers (aside from BigInt) are **IEEE-754 double-precision 64-bit floats:** 1 sign bit, 11 exponent bits, 52 mantissa bits.

Consequences:

```
0.1 + 0.2;                  // 0.30000000000000004
0.1 + 0.2 === 0.3;          // false
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON;  // true — how to compare

Number.MAX_SAFE_INTEGER;    // 9007199254740991  (2^53 - 1)
Number.MIN_SAFE_INTEGER;    // -9007199254740991
9007199254740992 === 9007199254740993;  // true ← precision loss
Number.isSafeInteger(2**53); // false

Number.MAX_VALUE;           // 1.7976931348623157e+308
Number.MIN_VALUE;           // 5e-324 (smallest positive denormal)
Number.EPSILON;             // 2.220446049250313e-16
```

Special values:

```
Infinity;  -Infinity;       // overflow, or 1/0
NaN;                        // "Not a Number" — result of invalid math
NaN === NaN;                // false ← NaN is not equal to itself
Number.isNaN(NaN);          // true  (strict — the right check)
isNaN("foo");               // true  (global — coerces first, avoid)
Object.is(NaN, NaN);        // true

0 === -0;                   // true
Object.is(0, -0);           // false
1 / -0;                     // -Infinity ← how to detect -0
```

Integer representation: integers are exact up to 2^53−1. Bitwise operators convert to 32-bit signed ints (§5.5 p. 22), which is a separate and lossier domain.

2.6 String

Immutable sequences of UTF-16 code units. Details in §18 p. 70.

```
"hello"[0];         // "h"
"hello"[0] = "H";   // silently fails (throws in strict mode)
"hello".length;     // 5
"😀 ".length;        // 2 ← one astral character, two code units
```

2.7 Symbol

Unique, immutable primitive values used as property keys that can't collide. Details in §25 p. 90.

```
const s = Symbol("desc");
const t = Symbol("desc");
s === t;            // false — always unique
```

2.8 BigInt

Arbitrary-precision integers. Details in §20 p. 77.

```
const big = 9007199254740993n;
big + 1n;           // 9007199254740994n
// big + 1;         ← TypeError: cannot mix BigInt and Number
```

2.9 Object

Everything that is not a primitive. Includes plain objects, arrays, functions, dates, regexes, maps, sets, errors, promises, and every class instance.

```
typeof {};            // "object"
typeof [];            // "object"
typeof null;          // "object"
typeof function(){};  // "function"  ← special-cased
typeof class {};      // "function"
typeof new Date();    // "object"
```

2.10 Wrapper Objects (Boxing)

Primitives have no methods, yet `"abc".toUpperCase()` works. When you access a property on a primitive, the engine transiently wraps it in an object (`String`, `Number`, `Boolean`, `Symbol`, `BigInt`), reads the property, and discards the wrapper.

```
"abc".toUpperCase();      // works via a temporary String wrapper

const s = "abc";
s.custom = 1;             // wrapper created, prop set, wrapper discarded
s.custom;                 // undefined

new String("abc") === "abc";        // false — object vs. primitive
typeof new String("abc");           // "object"
Boolean(new Boolean(false));        // true ← every object is truthy!
```

Rule: never use `new String`, `new Number`, or `new Boolean`. Calling them *without* `new` is fine and is a normal conversion: `String(x)`, `Number(x)`, `Boolean(x)`.

2.11 Detecting Types Reliably

`typeof` is unreliable for objects. The most robust general-purpose check:

```
function typeOf(v) {
  return Object.prototype.toString.call(v).slice(8, -1);
}

typeOf(null);        // "Null"
typeOf([]);          // "Array"
typeOf(new Date());  // "Date"
typeOf(/x/);         // "RegExp"
typeOf(new Map());   // "Map"
typeOf(Promise.resolve()); // "Promise"
```

Purpose-built checks:

```
Array.isArray(v);
Number.isInteger(v);
Number.isFinite(v);
v instanceof Date;              // fails across realms/iframes
typeof v === "function";
v !== null && typeof v === "object";   // "is a non-null object"
Object.getPrototypeOf(v) === Object.prototype;  // "is a plain object"
```

CHAPTER 3

Variables, Scope, and Hoisting

IN THIS CHAPTER

3.1 Declaration Forms 11
3.2 `const` Immutability Is Shallow 11
3.3 Scope Types 12
3.4 The Scope Chain 12
3.5 Hoisting 12
3.6 The Temporal Dead Zone (TDZ) 13
3.7 Loops and Per-Iteration Bindings 13
3.8 Implicit Globals and `globalThis` 13
3.9 Shadowing 14

3.1 Declaration Forms

	`var`	`let`	`const`	`function`	`class`
Scope	function	block	block	function/block	block
Hoisted	yes, init `undefined`	yes, TDZ	yes, TDZ	yes, fully	yes, TDZ
Redeclarable	yes	no	no	yes	no
Reassignable	yes	yes	**no**	yes	yes
Creates global property	yes	no	no	yes (top-level)	no
Requires initializer	no	no	**yes**	—	—

```
var a = 1;      // function-scoped, legacy
let b = 2;      // block-scoped, mutable
const c = 3;    // block-scoped, immutable binding
```

3.2 `const` Immutability Is Shallow

`const` freezes the **binding**, not the value.

```
const obj = { a: 1 };
obj.a = 2;        // ✅ allowed — object contents are mutable
obj = {};         // ❌ TypeError: Assignment to constant variable

const arr = [1];
arr.push(2);      // ✅ [1, 2]

// For deep immutability:
Object.freeze(obj);            // shallow freeze
structuredClone(obj);          // deep copy (not freeze)
```

3.3 Scope Types

1. **Global scope** — outside any function or block.
2. **Module scope** — top level of an ES module; *not* global.
3. **Function scope** — created by every function; `var` lives here.
4. **Block scope** — created by `{ }`, `for`, `if`, `while`, etc.; `let`/`const`/`class` live here.

```
{
  var functionScoped = 1;
  let blockScoped = 2;
}
functionScoped;  // 1
blockScoped;     // ReferenceError
```

3.4 The Scope Chain

Identifier resolution walks outward from the innermost scope to the global scope. The chain is fixed **lexically** — by where code is *written*, not where it's called.

```
const x = "global";
function outer() {
  const x = "outer";
  function inner() {
    console.log(x);   // "outer" — nearest enclosing binding
  }
  return inner;
}
outer()();  // "outer", regardless of where it's invoked
```

3.5 Hoisting

Declarations are processed before any code runs in that scope.

```
console.log(v);   // undefined  ← declaration hoisted, assignment not
var v = 1;

console.log(l);   // ReferenceError: Cannot access 'l' before initialization
let l = 1;

hoisted();        // "works" ← whole function hoisted
function hoisted() { console.log("works"); }

notHoisted();     // TypeError: notHoisted is not a function
var notHoisted = function () {};
```

3.6 The Temporal Dead Zone (TDZ)

`let`, `const`, and `class` bindings exist from the start of the block but are *uninitialized* until the declaration executes. Touching them in between throws.

```
{
  // TDZ for `t` starts here
  typeof t;      // ReferenceError ← even typeof throws in the TDZ
  let t = 1;     // TDZ ends
  typeof t;      // "number"
}

// A subtle one:
let x = 1;
{
  console.log(x);  // ReferenceError, NOT 1 — inner `x` shadows from block start
  let x = 2;
}
```

3.7 Loops and Per-Iteration Bindings

`let` in a `for` loop creates a **fresh binding each iteration**; `var` does not.

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));  // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i));  // 0 1 2
```

Pre-ES6 workaround was an IIFE capturing the value:

```
for (var i = 0; i < 3; i++) (function (j) {
  setTimeout(() => console.log(j));
})(i);   // 0 1 2
```

3.8 Implicit Globals and `globalThis`

In sloppy mode, assigning to an undeclared identifier creates a property on the global object:

```
function f() { leaked = 1; }   // creates globalThis.leaked
f();
// In strict mode: ReferenceError — always what you want.
```

`globalThis` (ES2020) is the portable reference to the global object: `window` in browsers, `global` in Node, `self` in workers.

```
globalThis.foo = 1;

// Top-level `var` in a *script* creates a global property; `let` does not:
var a = 1;  // globalThis.a === 1
let b = 2;  // globalThis.b === undefined
// In a *module*, neither does.
```

3.9 Shadowing

An inner binding with the same name hides the outer one.

```
const x = 1;
function f() {
  const x = 2;   // shadows
  console.log(x);  // 2
}
```

Illegal shadowing: you cannot `let` a name in the same scope where a `var` of that name exists, or vice versa.

```
function f() {
  var y = 1;
  let y = 2;   // SyntaxError: Identifier 'y' has already been declared
}
```

CHAPTER 4

Type Coercion and Conversion

IN THIS CHAPTER

4.1 The Abstract Operations 15
4.2 ToBoolean 15
4.3 ToNumber 16
4.4 ToString 17
4.5 ToPrimitive and Hints 17
4.6 The + Operator 17
4.7 Loose Equality == — The Full Algorithm 18
4.8 Strict Equality === 19
4.9 Object.is — SameValue 19
4.10 Relational Comparison 19

This is where most JavaScript surprises live. Understanding the abstract operations demystifies nearly all of them.

4.1 The Abstract Operations

The spec defines conversions that operators invoke implicitly:

- **ToPrimitive(input, hint)** — object → primitive
- **ToBoolean(v)**
- **ToNumber(v)**
- **ToString(v)**
- **ToPropertyKey(v)**

4.2 ToBoolean

Exactly **8 falsy values.** Everything else is truthy.

```
false, 0, -0, 0n, "", null, undefined, NaN
```

```
Boolean([]);        // true  ← empty array is truthy
Boolean({});        // true
Boolean("0");       // true  ← non-empty string
Boolean("false");   // true
Boolean(new Boolean(false));  // true ← it's an object
Boolean(document.all);        // false ← the one legacy exception, browsers only
```

4.3 ToNumber

Input	Result
`undefined`	`NaN`
`null`	`0`
`true` / `false`	`1` / `0`
String	parsed numeric literal; `""` → `0`; invalid → `NaN`
Symbol	**TypeError**
BigInt	**TypeError** (in `Number()` it converts)
Object	`ToNumber(ToPrimitive(v, "number"))`

```
Number("");        // 0
Number("  12  ");  // 12   ← whitespace trimmed
Number("12px");    // NaN  ← unlike parseInt
Number("0x1F");    // 31
Number("1e3");     // 1000
Number("Infinity");// Infinity
Number([]);        // 0    ← [] → "" → 0
Number([5]);       // 5    ← [5] → "5" → 5
Number([1,2]);     // NaN  ← "1,2"
Number({});        // NaN  ← "[object Object]"
Number(null);      // 0
Number(undefined); // NaN

parseInt("12px");    // 12   ← stops at first invalid char
parseInt("px12");    // NaN
parseInt("08");      // 8
parseInt("0x1F");    // 31
parseInt("1e3");     // 1    ← does not understand exponents!
parseFloat("1e3");   // 1000
parseInt(0.0000005); // 5 ← because String(0.0000005) === "5e-7"
```

`parseInt` takes a radix — **always pass it:** `parseInt(str, 10)`.

4.4 ToString

```
String(null);        // "null"
String(undefined);   // "undefined"
String(123);         // "123"
String(-0);          // "0"      ← note
String(1e21);        // "1e+21"
String(Symbol("x")); // "Symbol(x)"  ← but `"" + sym` throws TypeError
String([]);          // ""
String([1,[2,[3]]]); // "1,2,3"   ← flattening join
String([null]);      // ""        ← null/undefined become empty in join
String({});          // "[object Object]"
String(function f(){});  // full source text
```

4.5 ToPrimitive and Hints

When an object must become a primitive, the engine calls:

1. `obj[Symbol.toPrimitive](hint)` if present
2. otherwise, for hint `"number"` or `"default"`: `valueOf()` then `toString()`
3. otherwise, for hint `"string"`: `toString()` then `valueOf()`

Whichever returns a primitive first wins; if neither does, **TypeError**.

Hints used: `"string"` (template literals, `String()`, property keys), `"number"` (arithmetic, comparison, `Number()`), `"default"` (`+` and `==`).

```
const money = {
  amount: 42,
  [Symbol.toPrimitive](hint) {
    if (hint === "number") return this.amount;
    if (hint === "string") return `$${this.amount}`;
    return `Money(${this.amount})`;   // default
  }
};

+money;          // 42
`${money}`;      // "$42"
money + "";      // "Money(42)"
```

Why `Date` is weird: `Date.prototype[Symbol.toPrimitive]` treats the `"default"` hint as `"string"`, so `date + 1` concatenates while `date - 1` subtracts.

4.6 The + Operator

`+` is overloaded: if **either** operand becomes a string after ToPrimitive, it concatenates; otherwise it adds numerically.

```
1 + 2;          // 3
"1" + 2;        // "12"
1 + "2";        // "12"
1 + 2 + "3";    // "33"    ← left to right
"1" + 2 + 3;    // "123"
1 + true;       // 2
1 + null;       // 1
1 + undefined;  // NaN
[] + [];        // ""      ← "" + ""
[] + {};        // "[object Object]"
{} + [];        // 0       ← in statement position, {} is a BLOCK, then +[] → 0
({}) + [];      // "[object Object]"
"5" - 3;        // 2       ← only `+` concatenates; -,*,/,% always numeric
"5" * "2";      // 10
```

4.7 Loose Equality == — The Full Algorithm

For `x == y`:

1. Same type → use `===`.
2. `null == undefined` → **true**. (Either with anything else → false.)
3. Number vs String → convert the string to a number.
4. BigInt vs String → convert the string to BigInt; if invalid → false.
5. Boolean on either side → convert it to a number, then restart.
6. Number/BigInt/String/Symbol vs Object → `ToPrimitive(object)`, then restart.
7. BigInt vs Number → compare mathematical values (no precision loss).
8. Otherwise → false.

```
0 == "";           // true   (string → 0)
0 == "0";          // true
"" == "0";         // false  (both strings, not equal)
0 == false;        // true
"0" == false;      // true   (false→0, "0"→0)
null == 0;         // false  ← null only loosely equals undefined
null == undefined; // true
NaN == NaN;        // false
[] == false;       // true   ([]→""→0, false→0)
[] == ![];         // true   (![] is false → 0; [] → 0)
[1] == 1;          // true
[[]] == 0;         // true
"1" == 1n;         // true
```

Rule: use `===`. The single defensible `==` is `x == null`, which tests "null or undefined" in one shot.

4.8 Strict Equality ===

Same type required, then:

- `NaN === NaN` → false
- `+0 === -0` → true
- Objects compared by reference identity

4.9 `Object.is` — SameValue

```
Object.is(NaN, NaN);   // true
Object.is(0, -0);      // false
Object.is(1, 1);       // true
```

Three equality semantics exist in the language:

Semantics	Where used
Loose (`==`)	`==`, `!=`
Strict (`===`)	`===`, `!==`, `switch`, `Array.prototype.indexOf`
SameValueZero	`Array.prototype.includes`, `Map` / `Set` keys (NaN equals NaN, ±0 equal)
SameValue	`Object.is`, `Object.defineProperty`

4.10 Relational Comparison

`<`, `>`, `<=`, `>=` use ToPrimitive with hint `"number"`. If **both** results are strings, compare lexicographically by UTF-16 code unit; otherwise compare numerically.

```
"10" < "9";     // true  ← string comparison
10 < "9";       // false ← numeric
"a" < "b";      // true
"Z" < "a";      // true  ← uppercase code units are lower
NaN < 1;        // false
NaN >= 1;       // false ← all comparisons with NaN are false
null >= 0;      // true   ← null → 0
null > 0;       // false
null == 0;      // false  ← inconsistent with the above, by design
```

For human-correct string ordering use `a.localeCompare(b)` or `Intl.Collator`.

CHAPTER 5

Operators and Expressions

IN THIS CHAPTER

5.1 Operator Precedence Table 20
5.2 Arithmetic 21
5.3 Assignment Operators 21
5.4 Comparison Operators 22
5.5 Bitwise Operators 22
5.6 Logical Operators and Short-Circuiting 23
5.7 Optional Chaining `?.` (ES2020) 23
5.8 Conditional (Ternary) Operator 23
5.9 `typeof`, `void`, `delete` 24
5.10 The Comma Operator 24
5.11 Grouping and Expression Statements 24
5.12 Spread and Rest `...` 24
5.13 Tagged Template Literals 24

5.1 Operator Precedence Table

Highest to lowest. Associativity: L = left-to-right, R = right-to-left.

Prec	Operator	Assoc
18	`(...)` grouping	—
17	`a.b`, `a?.b`, `a[b]`, `f()`, `new X(...)`, `tag ```	L
16	`new X` (no args)	R
15	`a++`, `a--` (postfix)	—
14	`!`, `~`, `+`, `-`, `++a`, `--a`, `typeof`, `void`, `delete`, `await`	R
13	`**`	**R**
12	`*`, `/`, `%`	L
11	`+`, `-`	L
10	`<<`, `>>`, `>>>`	L
9	`<`, `<=`, `>`, `>=`, `in`, `instanceof`	L
8	`==`, `!=`, `===`, `!==`	L
7	`&`	L
6	`^`	L
5	`|`	L

Prec	Operator	Assoc
4	`&&`	L
3	`||`, `??`	L
2	`? :`, `=`, `+=`, `&&=`, `||=`, `??=`, `...`, `yield`, `yield*`	R
1	`,`	L

Notes:

- `**` is right-associative: `2 ** 3 ** 2` → `2 ** 9` → 512.
- `-2 ** 2` is a **SyntaxError** — you must write `(-2) ** 2`.
- `??` cannot be mixed with `&&`/`||` without parentheses: `a || b ?? c` is a SyntaxError.

5.2 Arithmetic

```
5 + 2;    // 7
5 - 2;    // 3
5 * 2;    // 10
5 / 2;    // 2.5  ← always float division
5 % 2;    // 1
-5 % 2;   // -1   ← sign follows the DIVIDEND (remainder, not modulo)
5 ** 2;   // 25
```

True modulo: `((a % n) + n) % n`.

Increment/decrement:

```
let i = 5;
i++;   // returns 5, i becomes 6   (postfix)
++i;   // i becomes 7, returns 7   (prefix)
```

Unary `+` and `-` coerce to number: `+"5"` → `5`, `-"5"` → `-5`, `+true` → `1`.

5.3 Assignment Operators

```
x = y
x += y    x -= y    x *= y    x /= y    x %= y    x **= y
x <<= y   x >>= y   x >>>= y
x &= y    x ^= y    x |= y
x &&= y   x ||= y   x ??= y     // logical assignment, ES2021
```

Logical assignment **short-circuits** — it only assigns (and only triggers setters) when needed:

```
a ||= b;   // a = a || b   → assigns if a is falsy
a &&= b;   // a = a && b   → assigns if a is truthy
a ??= b;   // a = a ?? b   → assigns if a is null/undefined

opts.retries ??= 3;   // default only when absent, preserves 0
```

Assignment is an expression and returns the assigned value: `let a, b; a = b = 5;`

5.4 Comparison Operators

Covered in §4.7 p. 18–4.10. Also:

```
"a" in { a: 1 };        // true  — own or inherited property
0 in [1, 2];            // true  — index 0 exists
2 in [1, 2];            // false — length 2, indices 0 and 1
"length" in [];         // true

[] instanceof Array;    // true
[] instanceof Object;   // true — walks the prototype chain
```

`instanceof` is customizable via `Symbol.hasInstance`, and fails across realms (an array from an iframe is not `instanceof` your `Array`). `Array.isArray` works across realms.

5.5 Bitwise Operators

Operands are converted to **32-bit signed integers** (except `>>>`, which yields unsigned). Anything above $2^{31}-1$ wraps.

```
5 & 3;      // 1     AND      0101 & 0011 = 0001
5 | 3;      // 7     OR
5 ^ 3;      // 6     XOR
~5;         // -6    NOT      ~n === -(n+1)
5 << 1;     // 10    left shift (×2)
5 >> 1;     // 2     signed right shift (÷2, keeps sign)
-5 >> 1;    // -3
-5 >>> 1;   // 2147483645   unsigned right shift, fills with 0
```

Idioms:

```
n | 0;        // truncate toward zero (32-bit only)
~~n;          // same
n >>> 0;      // to uint32
~arr.indexOf(x)   // truthy when found (indexOf -1 → ~-1 === 0)
flags & MASK      // test a bit flag
```

Caution: 32-bit truncation makes these unsafe for large numbers. `Math.trunc` is clearer and correct.

5.6 Logical Operators and Short-Circuiting

`&&` and `||` return **one of their operands**, not a boolean.

```
"a" && "b";      // "b"   — returns last if all truthy
0 && "b";        // 0     — returns first falsy
"a" || "b";      // "a"   — returns first truthy
null || "b";     // "b"
!"a";            // false — `!` always returns a real boolean

const name = input || "anonymous";   // falls back on "" and 0 too!
const name = input ?? "anonymous";   // falls back only on null/undefined
```

`??` (nullish coalescing, ES2020) checks only `null` and `undefined`.

5.7 Optional Chaining `?.` (ES2020)

Short-circuits to `undefined` if the left side is `null` or `undefined`.

```
obj?.prop
obj?.[expr]
fn?.()
obj?.method?.()
arr?.[0]?.name

a?.b.c        // if a is nullish → undefined (whole chain short-circuits)
(a?.b).c      // parentheses BREAK the short circuit → TypeError
delete obj?.prop;   // legal
// obj?.prop = 1;   ← SyntaxError: not valid on the left of assignment
```

5.8 Conditional (Ternary) Operator

```
const label = count === 1 ? "item" : "items";

// Chained (right-associative):
const size = n < 10 ? "S" : n < 100 ? "M" : "L";
```

5.9 `typeof`, `void`, `delete`

```
typeof undeclaredVar;   // "undefined" — the ONLY safe way to touch an undeclared
name
void 0;                 // undefined
void fn();              // evaluates fn(), yields undefined

const o = { a: 1 };
delete o.a;             // true, removes the own property
delete o.nope;          // true (no-op)
delete Math.PI;         // false — non-configurable (throws in strict mode)
const arr = [1,2,3];
delete arr[1];          // leaves a HOLE: [1, <empty>, 3], length still 3
```

`delete` removes properties only — it does not free memory or delete variables.

5.10 The Comma Operator

Evaluates left to right, returns the last value.

```
const x = (1, 2, 3);          // 3
for (let i = 0, j = 10; i < j; i++, j--) {}
```

5.11 Grouping and Expression Statements

```
(function () {})();    // IIFE
(() => {})();
!function () {}();     // alternate IIFE prefix
void function () {}();
```

5.12 Spread and Rest `...`

Contextual: **spread** expands, **rest** collects. See §15 p. 61.

5.13 Tagged Template Literals

```
function tag(strings, ...values) {
  // strings: array of literal chunks, with `.raw` for unescaped text
  return strings.raw.join("|");
}
tag`a${1}b\n`;   // "a|b\\n"

String.raw`C:\new\table`;   // "C:\new\table" — backslashes preserved
```

Used by libraries for SQL escaping, styled-components, i18n, and GraphQL.

CHAPTER 6

Statements and Control Flow

IN THIS CHAPTER

6.1 Blocks and Empty Statements 25
6.2 Conditionals 25
6.3 Loops 26
6.4 `break`, `continue`, and Labels 27
6.5 `try` / `catch` / `finally` / `throw` 27
6.6 `with` (Deprecated) 27
6.7 `debugger` 28
6.8 Declarations as Statements 28

6.1 Blocks and Empty Statements

```
{ let a = 1; }     // block — creates block scope
;                  // empty statement
while (poll()) ;   // deliberate empty body
if (x);            // ← almost always a bug
```

6.2 Conditionals

```
if (cond) { ... }
else if (other) { ... }
else { ... }
```

```
switch (value) {           // uses === , NO coercion
  case "a":
  case "b":                // fallthrough — shared body
    doAB();
    break;
  case compute():          // case expressions are evaluated in order
    doC();
    break;
  default:                 // may appear anywhere; runs if nothing matched
    doDefault();
}
```

`switch` gotchas:

- Forgetting `break` falls through to the next case.
- `switch (true)` with boolean `case` expressions is a common pattern for ranges.

- `let`/`const` inside a `switch` share one block scope across all cases — wrap a case in `{ }` to isolate.
- `switch (x)` with `x = NaN` never matches, since `NaN !== NaN`.

6.3 Loops

```
for (let i = 0; i < n; i++) { }
for (;;) { }                          // infinite

while (cond) { }
do { } while (cond);                  // body runs at least once

for (const key in obj) { }            // enumerable keys, INCLUDING inherited
for (const val of iterable) { }       // values from an iterator (ES2015)
for await (const v of asyncIterable) { }   // ES2018
```

`for...in` vs `for...of`:

	`for...in`	`for...of`
Iterates	enumerable string keys	iterable values
Includes inherited	yes	n/a
Order	integer-like keys ascending, then insertion order	iterator order
Works on plain objects	yes	no (unless it has `Symbol.iterator`)
Works on arrays	yes, but keys are **strings** and holes are skipped	yes, recommended

```
for (const i in [10, 20]) console.log(typeof i);  // "string" "string"
for (const v of [10, 20]) console.log(v);         // 10 20

// Safe for...in:
for (const k in obj) {
  if (!Object.hasOwn(obj, k)) continue;   // ES2022; older:
Object.prototype.hasOwnProperty.call
}

// Index + value:
for (const [i, v] of arr.entries()) { }
Object.entries(obj).forEach(([k, v]) => {});
```

6.4 break, continue, and Labels

```
outer:
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer;   // next i
    if (i === 2) break outer;      // exit both loops
  }
}

block: {                       // labeled block
  if (skip) break block;       // jump to the end
  doWork();
}
```

`break`/`continue` cannot cross a function boundary — that's why you cannot break out of `forEach` (use `for...of`, `some`, or `every`).

6.5 try / catch / finally / throw

```
try {
  risky();
} catch (err) {          // binding optional since ES2019: `catch {`
  handle(err);
} finally {
  cleanup();             // ALWAYS runs
}

throw new Error("boom");     // throw anything, but always throw an Error
```

`finally` semantics: it runs on normal completion, on exception, and on `return`/`break`/`continue`. A `return` inside `finally` **overrides** any pending return or exception — a notorious bug source.

```
function f() {
  try { return 1; }
  finally { return 2; }   // returns 2, silently swallowing the 1
}
```

6.6 with (Deprecated)

```
with (obj) { a = b; }   // resolves a,b against obj first — forbidden in strict mode
```

Never use it. It defeats static scoping and optimization.

6.7 debugger

```
debugger;   // breakpoint if a debugger is attached; otherwise a no-op
```

6.8 Declarations as Statements

`function`, `class`, `var`, `let`, `const`, `import`, `export` are declarations. Note that a function declaration is not allowed as the sole body of an `if` in strict mode:

```
if (x) function f() {}    // legal in sloppy (Annex B), SyntaxError in strict
if (x) { function f() {} } // fine
```

PART II

Functions and Objects

7	Functions	30
8	Closures	36
9	this and Execution Context	38
10	Objects	41
11	Property Descriptors and Object Meta-Operations	46
12	Prototypes and Inheritance	48
13	Classes	51

CHAPTER 7

Functions

IN THIS CHAPTER

7.1 The Five Ways to Define a Function 30
7.2 Parameters 31
7.3 The `arguments` Object 32
7.4 Return Values 32
7.5 Arrow Functions in Detail 32
7.6 Immediately Invoked Function Expressions (IIFE) 33
7.7 Recursion 33
7.8 Higher-Order Functions 34
7.9 `call`, `apply`, `bind` 34
7.10 Function Properties and Methods 35
7.11 `new.target` 35

Functions are **first-class objects**: assignable, passable, returnable, and property-bearing.

7.1 The Five Ways to Define a Function

```
// 1. Function declaration — hoisted entirely
function add(a, b) { return a + b; }

// 2. Function expression — not hoisted
const add = function (a, b) { return a + b; };

// 3. Named function expression — name visible only inside (useful for recursion/
stacks)
const fact = function f(n) { return n < 2 ? 1 : n * f(n - 1); };

// 4. Arrow function — lexical `this`, no `arguments`, not constructible
const add = (a, b) => a + b;

// 5. Function constructor — compiles a string, global scope only, avoid
const add = new Function("a", "b", "return a + b");
```

Also: method shorthand, getters/setters, generators, async variants:

```
const obj = {
  method() {},
  *gen() {},
  async load() {},
  async *stream() {},
  get value() { return 1; },
  set value(v) {},
};
```

7.2 Parameters

```
function f(a, b = 10, ...rest) { }   // default + rest
```

Default parameters (ES2015):

```
function greet(name = "world", greeting = `hello ${name}`) {
  // defaults are evaluated at CALL time, left to right,
  // and later defaults can reference earlier params
  return `${greeting}!`;
}
greet();            // "hello world!"
greet(undefined);   // default applies — ONLY undefined triggers it
greet(null);        // null is passed through, no default

function required(name) { throw new Error(`${name} is required`); }
function f(x = required("x")) {}   // enforce required args
```

Any parameter list containing defaults, rest, or destructuring makes the function's parameter scope separate and disables the `arguments` object's aliasing.

Rest parameters collect remaining arguments into a real array; must be last and cannot have a default.

Options-object pattern:

```
function connect({ host = "localhost", port = 80, secure = false } = {}) { }
connect();
connect({ port: 8080 });
```

`length` **and** `name`
:

```
function f(a, b = 1, ...c) {}
f.length;   // 1 — counts params BEFORE the first default/rest
f.name;     // "f"
((a) => a).name;              // ""  (anonymous, unless inferred)
const g = (a) => a; g.name;   // "g" (inferred from assignment)
```

7.3 The `arguments` Object

An array-like (not an array) available in non-arrow functions.

```
function f() {
  arguments.length;
  arguments[0];
  Array.from(arguments);       // convert
  [...arguments];              // convert
  Array.prototype.slice.call(arguments);  // legacy convert
}
```

In sloppy mode with simple parameters, `arguments[i]` is *aliased* to the named parameter — writing one changes the other. Strict mode removes this. Prefer rest parameters; `arguments` is legacy and deoptimizes some engines.

`arguments.callee` and `arguments.caller` are forbidden in strict mode.

7.4 Return Values

Every function returns something: an explicit value, or `undefined`. A constructor called with `new` returns `this` unless it explicitly returns an object.

```
function C() { this.a = 1; return 5; }        // returns the instance (primitive
ignored)
function D() { this.a = 1; return { b: 2 }; } // returns { b: 2 }
```

7.5 Arrow Functions in Detail

```
x => x * 2                 // one param, implicit return
(x, y) => x + y
() => 42
x => ({ value: x })        // parenthesize an object literal body
x => { return x; }         // block body needs explicit return
async x => await f(x)
```

Differences from normal functions:

Feature	Normal	Arrow
`this`	dynamic, call-site determined	**lexical**, inherited
`arguments`	own object	inherited from enclosing scope
`new`	constructible (unless method)	**TypeError**
`prototype` property	yes	no
`super` / `new.target`	own	lexical

Feature	Normal	Arrow
Generator	can be	cannot be
Hoisting	declarations hoist	expression only

```
// The canonical use:
class Timer {
  constructor() { this.n = 0; }
  start() {
    setInterval(() => { this.n++; }, 1000);     // ✅  `this` is the Timer
    setInterval(function () { this.n++; }, 1000); // ❌  `this` is
undefined/global
  }
}

// The canonical misuse:
const obj = {
  name: "x",
  bad: () => this.name,       // `this` is module/global scope, NOT obj
  good() { return this.name; }
};
```

7.6 Immediately Invoked Function Expressions (IIFE)

```
(function () { /* private scope */ })();
(function () { })();
(() => { })();
(async () => { await main(); })();   // top-level await workaround in scripts
```

Historically used for module privacy and to avoid globals. Largely replaced by block scope and ES modules, but still used for async entry points.

7.7 Recursion

```
function fact(n) { return n <= 1 ? 1 : n * fact(n - 1); }
```

Engines impose a call-stack limit (typically ~10k frames) → `RangeError: Maximum call stack size exceeded`. Proper tail calls are in the spec but only shipped in JavaScriptCore/Safari. Convert deep recursion to iteration or use an explicit stack, or trampoline:

```
function trampoline(fn) {
  return (...args) => { let r = fn(...args); while (typeof r === "function") r =
r(); return r; }; }
```

7.8 Higher-Order Functions

Functions that take and/or return functions.

```
const twice = f => x => f(f(x));
const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x);
const pipe    = (...fns) => x => fns.reduce((v, f) => f(v), x);

const curry = fn => function c(...a) {
  return a.length >= fn.length ? fn(...a) : (...b) => c(...a, ...b);
};

const memoize = fn => {
  const cache = new Map();
  return (...a) => {
    const k = JSON.stringify(a);
    if (!cache.has(k)) cache.set(k, fn(...a));
    return cache.get(k);
  };
};

const debounce = (fn, ms) => {
  let t;
  return (...a) => { clearTimeout(t); t = setTimeout(() => fn(...a), ms); };
};

const throttle = (fn, ms) => {
  let last = 0;
  return (...a) => { const now = Date.now(); if (now - last >= ms) { last = now;
fn(...a); } };
};

const once = fn => { let done = false, val; return (...a) => done ? val : (done =
true, val = fn(...a)); };
```

7.9 `call`, `apply`, `bind`

```
fn.call(thisArg, a, b);          // args listed
fn.apply(thisArg, [a, b]);       // args as array
const bound = fn.bind(thisArg, a);   // returns a new permanently-bound function
```

- `bind` supports partial application; the bound `this` **cannot** be overridden later, even by another `bind` or `call`.
- `bound.name` is `"bound fn"`; `bound.length` is the original minus bound args.
- `new BoundFn()` ignores the bound `this` but keeps bound arguments.

```
// Borrowing methods:
Array.prototype.slice.call(arrayLike);
Object.prototype.hasOwnProperty.call(obj, key);
Function.prototype.call.bind(Object.prototype.hasOwnProperty);  // uncurried
```

7.10 Function Properties and Methods

```
fn.length      // declared arity
fn.name        // function name
fn.prototype   // object used as [[Prototype]] for instances (absent on arrows/
methods)
fn.constructor // Function
fn.toString()  // source text (ES2019 guarantees exact source)
```

7.11 `new.target`

Detects whether a function was invoked with `new`.

```
function C() {
  if (new.target === undefined) throw new TypeError("Use new");
  // in a subclass call, new.target is the subclass
}
```

CHAPTER 8

Closures

IN THIS CHAPTER

8.1 The Mechanism 36
8.2 Uses 36
8.3 Classic Pitfalls 37

A **closure** is a function together with the lexical environment in which it was created. Every JavaScript function is a closure; the term is usually reserved for cases where the inner function outlives the outer call.

8.1 The Mechanism

```
function makeCounter() {
  let count = 0;                // lives in makeCounter's environment
  return {
    inc: () => ++count,
    get: () => count,
  };
}
const c = makeCounter();
c.inc(); c.inc();
c.get();   // 2 — `count` survived because the closures reference it
```

The environment is kept alive as long as *any* closure references it. Each call to `makeCounter` creates an **independent** environment.

8.2 Uses

Data privacy / module pattern:

```
const store = (() => {
  const data = new Map();                    // truly private
  return { set: (k, v) => data.set(k, v), get: k => data.get(k) };
})();
```

Function factories:

```
const multiplier = n => x => x * n;
const double = multiplier(2);
```

Stateful callbacks, memoization, partial application, event handlers, iterators — all built on closures.

8.3 Classic Pitfalls

Loop variable capture (see §3.7 p. 13): `var` shares one binding.

Accidental retention: a closure keeps the *entire* environment record alive in most engines, so a single small closure can pin a large object in memory.

```
function setup() {
  const huge = new Array(1e6).fill(0);
  const small = huge.length;
  return () => small;      // may still retain `huge` depending on engine
optimization
}
// Defensive: null out large refs, or narrow the closure's scope with an inner IIFE.
```

Shared vs. copied state: closures capture *variables*, not values.

```
let x = 1;
const f = () => x;
x = 2;
f();   // 2, not 1
```

CHAPTER 9

`this` and Execution Context

IN THIS CHAPTER

9.1 The Binding Rules, in Priority Order 38
9.2 Losing `this` 38
9.3 `this` in Various Contexts 39
9.4 `super` 39

`this` is determined by **how a function is called**, not where it is defined — except for arrow functions.

9.1 The Binding Rules, in Priority Order

1. `new` **binding** — `new Fn()` → `this` is the freshly created object.
2. **Explicit binding** — `fn.call(o)`, `fn.apply(o)`, `fn.bind(o)` → `this` is `o`.
3. **Implicit binding** — `o.fn()` → `this` is `o` (the object *immediately* left of the dot).
4. **Default binding** — plain `fn()` → `undefined` in strict mode, `globalThis` in sloppy.

Arrow functions bypass all four and use the enclosing lexical `this`.

```
function show() { return this; }
const o = { show };

new show();          // the new object
show.call(o);        // o
o.show();            // o
show();              // undefined (strict) / globalThis (sloppy)
```

9.2 Losing `this`

Extracting a method detaches it from its receiver.

```
const btn = { label: "OK", click() { return this.label; } };
const handler = btn.click;
handler();                      // TypeError / undefined — `this` lost

// Fixes:
btn.click.bind(btn);
() => btn.click();
element.addEventListener("click", () => btn.click());
```

Class fields as arrow functions permanently bind per-instance:

```
class Button {
  label = "OK";
  click = () => this.label;    // bound to the instance, safe to pass around
}
```

9.3 `this` in Various Contexts

Context	Value of `this`
Global scope, script, sloppy	`globalThis`
Global scope, script, strict	`globalThis`
Module top level	`undefined`
Plain function call, strict	`undefined`
Plain function call, sloppy	`globalThis`
Method call	the receiver
Constructor with `new`	the new instance
Class body / methods	the instance (class bodies are always strict)
Static method	the class itself
Arrow function	lexical, from enclosing scope
Event handler (DOM, `function`)	the element
`setTimeout(fn)` sloppy	`globalThis` (Node: `Timeout` object)
Getter/setter	the receiver

9.4 `super`

Available in methods of classes and object literals with a prototype.

```
class A { greet() { return "A"; } }
class B extends A { greet() { return super.greet() + "B"; } }
```

`super` is resolved from the method's `[[HomeObject]]`, which is why it works in shorthand methods but not in function-expression properties.

CHAPTER 10

Objects

IN THIS CHAPTER

10.1 Creating Objects 41
10.2 Object Literal Syntax 41
10.3 Accessing Properties 42
10.4 Setting and Deleting 42
10.5 Property Key Types and Ordering 42
10.6 Enumerating Properties 43
10.7 Checking for Properties 43
10.8 Copying and Merging 44
10.9 Comparing Objects 44
10.10 Getters and Setters (Accessor Properties) 44
10.11 Object Statics Reference 44

An object is a collection of properties, each with a key (string or symbol) and a value, plus an internal `[[Prototype]]` link.

10.1 Creating Objects

```
const o = {};                          // literal
const o = new Object();                // equivalent, verbose
const o = Object.create(proto);        // explicit prototype
const o = Object.create(null);         // NO prototype — a clean dictionary
const o = new MyClass();               // constructor
const o = Object.fromEntries(pairs);   // from [k,v] pairs (ES2019)
const o = structuredClone(other);      // deep clone
```

`Object.create(null)` objects have no `toString`, `hasOwnProperty`, or `__proto__` — ideal as a map when you can't use `Map`, because no key can collide with an inherited name.

10.2 Object Literal Syntax

```
const key = "dynamic";
const value = 1;

const o = {
  plain: 1,
  "with spaces": 2,
  42: 3,                    // numeric keys become strings
  value,                    // shorthand property (ES2015)
  [key]: 4,                 // computed key (ES2015)
```

```
  [`${key}_2`]: 5,
  [Symbol.iterator]: fn,
  method() {},               // method shorthand
  get computed() { return 1; },
  set computed(v) {},
  ...otherObject,            // spread (ES2018)
  __proto__: base,           // sets the prototype (literal only, special-cased)
};
```

Trailing commas are allowed. Duplicate keys: the last wins (SyntaxError in strict mode only for ES5-era code; now allowed).

10.3 Accessing Properties

```
o.prop;             // dot — static identifier keys only
o["prop"];          // bracket — any expression, needed for dynamic/invalid-
identifier keys
o?.prop;            // optional chaining
o?.[expr];
o.prop ?? fallback;

const { prop } = o; // destructuring
```

Reading a missing property returns `undefined` (it does not throw). Reading a property of `null`/`undefined` throws `TypeError`.

10.4 Setting and Deleting

```
o.prop = 1;
o["prop"] = 1;
Object.assign(o, { a: 1, b: 2 });
Object.defineProperty(o, "x", { value: 1 });

delete o.prop;
```

Assignment consults the prototype chain for setters and for non-writable properties:

```
const proto = Object.freeze({ x: 1 });
const child = Object.create(proto);
child.x = 2;        // silently fails (strict: TypeError) — inherited non-writable
```

10.5 Property Key Types and Ordering

Keys are **strings or symbols**. Everything else is converted via `ToPropertyKey`.

```
const o = {};
o[1] = "a";      // key is "1"
o[true] = "b";   // key is "true"
o[{}] = "c";     // key is "[object Object]"
Object.keys(o);  // ["1", "true", "[object Object]"]
```

Enumeration order (deterministic since ES2015):

1. Integer-like string keys ("array indices"), ascending numeric order
2. Other string keys, in insertion order
3. Symbol keys, in insertion order

```
Object.keys({ b: 1, 2: 2, a: 3, 1: 4 });  // ["1", "2", "b", "a"]
```

This is why `Map` is preferable when key order must be pure insertion order.

10.6 Enumerating Properties

Method	Own	Inherited	Enumerable only	Strings	Symbols
`Object.keys(o)`	✅	❌	✅	✅	❌
`Object.values(o)`	✅	❌	✅	✅	❌
`Object.entries(o)`	✅	❌	✅	✅	❌
`for...in`	✅	✅	✅	✅	❌
`Object.getOwnPropertyNames(o)`	✅	❌	❌ (all)	✅	❌
`Object.getOwnPropertySymbols(o)`	✅	❌	❌ (all)	❌	✅
`Reflect.ownKeys(o)`	✅	❌	❌ (all)	✅	✅
`JSON.stringify`	✅	❌	✅	✅	❌
Spread `{...o}`	✅	❌	✅	✅	✅
`Object.assign`	✅	❌	✅	✅	✅

10.7 Checking for Properties

```
"key" in o;                              // own OR inherited
Object.hasOwn(o, "key");                 // own only (ES2022) — preferred
o.hasOwnProperty("key");                 // own only, breaks on Object.create(null)
Object.prototype.hasOwnProperty.call(o, "key");   // safe legacy form
o.key !== undefined;                     // false negative if value IS undefined
```

10.8 Copying and Merging

```
// Shallow:
const copy = { ...o };
const copy = Object.assign({}, o);
const merged = { ...defaults, ...options };     // later wins

// Deep:
const deep = structuredClone(o);        // handles
Map/Set/Date/RegExp/cycles/TypedArray
const deep = JSON.parse(JSON.stringify(o));  // loses undefined, functions, Symbol,
                                             // Infinity→null, Date→string, cycles
throw
```

`Object.assign` triggers setters on the target and getters on the source; spread only copies values. `structuredClone` cannot clone functions, DOM nodes, or property descriptors.

10.9 Comparing Objects

Objects compare by identity only.

```
{ a: 1 } === { a: 1 };   // false
const o = {}; o === o;   // true
```

Structural comparison requires a custom deep-equal, or `JSON.stringify` on key-sorted objects (fragile).

10.10 Getters and Setters (Accessor Properties)

```
const temp = {
  _c: 0,
  get f() { return this._c * 9 / 5 + 32; },
  set f(v) { this._c = (v - 32) * 5 / 9; },
};
temp.f = 212;   // invokes the setter
temp._c;        // 100
```

Accessors run on every read/write, are inherited normally, and appear as `{get, set, enumerable, configurable}` descriptors. A getter without a setter makes writes silently fail (throw in strict mode).

10.11 Object Statics Reference

```
Object.keys / values / entries / fromEntries
Object.assign(target, ...sources)
Object.create(proto, descriptors?)
```

```
Object.defineProperty / defineProperties
Object.getOwnPropertyDescriptor / getOwnPropertyDescriptors
Object.getOwnPropertyNames / getOwnPropertySymbols
Object.getPrototypeOf / setPrototypeOf
Object.freeze / isFrozen
Object.seal / isSealed
Object.preventExtensions / isExtensible
Object.is(a, b)
Object.hasOwn(o, k)                  // ES2022
Object.groupBy(items, fn)            // ES2024 → plain object of arrays
```

```
Object.groupBy([1,2,3,4], n => n % 2 ? "odd" : "even");
// { odd: [1,3], even: [2,4] }
```

CHAPTER 11

Property Descriptors and Object Meta-Operations

IN THIS CHAPTER

11.1 Descriptor Shapes 46
11.2 Defining Properties 47
11.3 Sealing Levels 47
11.4 Well-Known Symbols as Hooks 47

11.1 Descriptor Shapes

Every property is either a **data property** or an **accessor property**.

```
// Data descriptor
{ value: 1, writable: true, enumerable: true, configurable: true }

// Accessor descriptor
{ get: fn, set: fn, enumerable: true, configurable: true }
```

Attribute	Meaning
`value`	The stored value
`writable`	Can be reassigned
`enumerable`	Shows in `for...in`, `Object.keys`, spread, JSON
`configurable`	Can be deleted or have its descriptor changed
`get` / `set`	Accessor functions

Defaults differ by creation route:

```
const a = {}; a.x = 1;
Object.getOwnPropertyDescriptor(a, "x");
// { value: 1, writable: true, enumerable: true, configurable: true }

const b = {};
Object.defineProperty(b, "x", { value: 1 });
Object.getOwnPropertyDescriptor(b, "x");
// { value: 1, writable: false, enumerable: false, configurable: false }  ← all
false!
```

11.2 Defining Properties

```
Object.defineProperty(obj, "id", {
  value: 42,
  writable: false,
  enumerable: false,
  configurable: false,
});

Object.defineProperties(obj, {
  a: { value: 1, enumerable: true },
  b: { get() { return 2; } },
});

// Faithful copy INCLUDING accessors and non-enumerables:
const clone = Object.create(
  Object.getPrototypeOf(src),
  Object.getOwnPropertyDescriptors(src)
);
```

Once `configurable: false`, the only permitted change is `writable: true → false`.

11.3 Sealing Levels

Operation	Add props	Delete props	Modify values	Reconfigure
(normal)	✅	✅	✅	✅
`preventExtensions`	❌	✅	✅	✅
`seal`	❌	❌	✅	❌
`freeze`	❌	❌	❌	❌

```
const o = Object.freeze({ a: 1, nested: { b: 2 } });
o.a = 9;          // silently ignored (TypeError in strict mode)
o.nested.b = 9;   // ✅ works — freeze is SHALLOW

function deepFreeze(o) {
  for (const v of Object.values(o)) if (v && typeof v === "object") deepFreeze(v);
  return Object.freeze(o);
}
```

`Object.freeze` on an array prevents push/pop/index assignment.

11.4 Well-Known Symbols as Hooks

Covered in §25.3 p. 90 — these let you customize `instanceof`, iteration, coercion, `Object.prototype.toString`, and more.

CHAPTER 12

Prototypes and Inheritance

IN THIS CHAPTER

12.1 The Prototype Chain 48
12.2 Accessing and Setting the Prototype 48
12.3 `prototype` vs `[[Prototype]]` 49
12.4 What `new` Actually Does 49
12.5 Constructor Function Inheritance (Pre-ES6) 49
12.6 The Full Built-in Chain 50
12.7 Mixins and Composition 50
12.8 Prototype Pollution (Security) 50

12.1 The Prototype Chain

Every object has an internal `[[Prototype]]` slot pointing to another object or `null`. Property lookup walks this chain.

```
const animal = { speak() { return "..."; } };
const dog = Object.create(animal);
dog.bark = () => "woof";

dog.speak();                        // found on animal
Object.getPrototypeOf(dog) === animal;   // true
Object.getPrototypeOf(animal) === Object.prototype;  // true
Object.getPrototypeOf(Object.prototype); // null — end of chain
```

Assignment does **not** walk the chain (except for setters) — it creates an own property, shadowing the inherited one.

12.2 Accessing and Setting the Prototype

```
Object.getPrototypeOf(o);
Object.setPrototypeOf(o, proto);    // legal but slow — deoptimizes the object
o.__proto__;                        // legacy accessor, Annex B, avoid
Object.create(proto);               // preferred way to set at creation
```

Mutating a prototype after object creation forces engines to discard hidden-class optimizations. Set it once, at creation.

12.3 `prototype` vs `[[Prototype]]`

This is the single most confused point in the language.

- `[[Prototype]]` (`__proto__`, `Object.getPrototypeOf`) — the link an object uses for lookup.
- `.prototype` — a property that exists only on *constructor functions*; it is the object that will become the `[[Prototype]]` of instances created with `new`.

```
function Dog(name) { this.name = name; }
Dog.prototype.speak = function () { return `${this.name} barks`; };

const rex = new Dog("Rex");
Object.getPrototypeOf(rex) === Dog.prototype;   // true
rex.__proto__ === Dog.prototype;                // true
rex.prototype;                                  // undefined — instances have none
Dog.prototype.constructor === Dog;              // true
```

12.4 What `new` Actually Does

```
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);   // 1. new object, prototype linked
  const result = Ctor.apply(obj, args);        // 2. run constructor with this = obj
  return (result !== null && typeof result === "object") ? result : obj;  // 3.
}
```

12.5 Constructor Function Inheritance (Pre-ES6)

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function () { return `${this.name} makes a noise`; };

function Dog(name, breed) {
  Animal.call(this, name);          // 1. inherit instance properties
  this.breed = breed;
}
Dog.prototype = Object.create(Animal.prototype);   // 2. inherit methods
Dog.prototype.constructor = Dog;                   // 3. restore constructor
Dog.prototype.speak = function () {                // 4. override
  return Animal.prototype.speak.call(this) + " (barks)";
};
```

Common mistake: `Dog.prototype = Animal.prototype` — now the two share one object and any change to `Dog.prototype` corrupts `Animal`.

12.6 The Full Built-in Chain

```
[]     → Array.prototype    → Object.prototype → null
"s"    → String.prototype   → Object.prototype → null
f()    → Function.prototype → Object.prototype → null
class instance → Class.prototype → Base.prototype → Object.prototype → null
```

Functions are objects whose prototype is `Function.prototype`, which is itself a callable object. `Object` is a function, so `Object.__proto__ === Function.prototype`, and `Function.prototype.__proto__ === Object.prototype` — the chain is circular in the sense that each depends on the other, resolved by the spec bootstrapping.

12.7 Mixins and Composition

```
const Serializable = Base => class extends Base {
  serialize() { return JSON.stringify(this); }
};
const Comparable = Base => class extends Base {
  equals(o) { return this.id === o.id; }
};

class Model {}
class User extends Serializable(Comparable(Model)) {}
```

Simple object mixins:

```
Object.assign(MyClass.prototype, mixinA, mixinB);   // copies enumerable own props
```

12.8 Prototype Pollution (Security)

Writing to `__proto__` via user-controlled keys can modify `Object.prototype` for the whole program.

```
// UNSAFE deep merge:
merge({}, JSON.parse('{"__proto__": {"isAdmin": true}}'));
({}).isAdmin;   // true — every object in the app is now compromised
```

Mitigations: reject the keys `__proto__`, `constructor`, `prototype`; use `Object.create(null)` or `Map` for user data; `Object.freeze(Object.prototype)`; use `JSON.parse` with a reviver that drops those keys.

CHAPTER 13

Classes

IN THIS CHAPTER

13.1 Declaration and Expression 51
13.2 Full Anatomy 51
13.3 Fields vs. Prototype Methods 52
13.4 Private Members # 53
13.5 Inheritance with `extends` and `super` 53
13.6 Extending Built-ins 54
13.7 Abstract Class Pattern 54
13.8 Class Limitations 54

Classes are syntax over the prototype system, plus genuinely new capabilities (private fields, `super`, correct subclassing of built-ins).

13.1 Declaration and Expression

```
class Point { }                       // declaration — hoisted but in TDZ
const Point = class { };              // expression
const Point = class Named { };        // named class expression
export default class { }              // anonymous
```

Class bodies are **always strict mode**. Classes are not callable without `new`.

13.2 Full Anatomy

```
class Rectangle extends Shape {
  // --- public instance field (ES2022) ---
  width = 0;

  // --- private instance field (ES2022) ---
  #height = 0;

  // --- static fields ---
  static unit = "px";
  static #count = 0;

  // --- static initialization block (ES2022) ---
  static {
    Rectangle.registry = new Map();
  }
```

```
  constructor(w, h) {
    super();                    // MUST come before `this` in a derived class
    this.width = w;
    this.#height = h;
    Rectangle.#count++;
  }

  // --- prototype method ---
  area() { return this.width * this.#height; }

  // --- getter / setter ---
  get height() { return this.#height; }
  set height(v) {
    if (v < 0) throw new RangeError("negative");
    this.#height = v;
  }

  // --- static method ---
  static square(size) { return new this(size, size); }

  // --- private method ---
  #validate() { return this.width > 0; }

  // --- static private method ---
  static #internal() {}

  // --- computed member name ---
  [Symbol.iterator]() { }

  // --- generator / async members ---
  *points() { }
  async load() { }
  static async fetch() { }

  // --- bound arrow field (per-instance) ---
  handleClick = () => this.area();
}
```

13.3 Fields vs. Prototype Methods

	Instance field	Prototype method
Stored on	each instance	`Class.prototype`
Memory	one copy per instance	one shared copy
`this`	lexical if arrow	dynamic
Enumerable	yes (own)	**no**
Assigned when	during construction, in declaration order, before constructor body	at class definition

Field initializers run: for a base class, before the constructor body; for a derived class, immediately after `super()` returns.

13.4 Private Members

```
class Account {
  #balance = 0;
  deposit(n) { this.#balance += n; }
  static isAccount(o) { return #balance in o; }   // ergonomic brand check (ES2022)
}

const a = new Account();
a.#balance;         // SyntaxError — genuinely inaccessible, not a convention
Object.keys(a);     // []
JSON.stringify(a);  // "{}"
```

Private names are lexically scoped to the class body. Accessing a `#` field on an object that doesn't have it throws `TypeError`. They are not reflectable — `Object.getOwnPropertyNames` won't show them, and `Proxy` cannot intercept them.

13.5 Inheritance with `extends` and `super`

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a noise`; }
  static create(n) { return new this(n); }    // `this` is the subclass when
inherited
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);                        // required before `this`
    this.breed = breed;
  }
  speak() { return super.speak() + " (barks)"; }
  static create(n) { return super.create(n); }
}
```

Rules:

- `extends` accepts any expression that evaluates to a constructor or `null`.
- In a derived constructor, `this` is in TDZ until `super()` is called.
- If a derived class omits a constructor, the implicit one is `constructor(...args) { super(...args); }`.
- `extends null` creates a class whose instances have no prototype, but you cannot use `this` in its constructor.

- Static members are inherited: `Dog.__proto__ === Animal`.

Two chains exist:

```
Dog.prototype  →  Animal.prototype  →  Object.prototype   (instance methods)
Dog            →  Animal            →  Function.prototype (static methods)
```

13.6 Extending Built-ins

```
class MyArray extends Array {
  last() { return this[this.length - 1]; }
  static get [Symbol.species]() { return Array; }  // map/filter return plain Arrays
}

class ValidationError extends Error {
  constructor(msg, field) {
    super(msg);
    this.name = "ValidationError";
    this.field = field;
    Error.captureStackTrace?.(this, ValidationError);   // V8 only
  }
}
```

`Symbol.species` controls what constructor derived methods (`map`, `filter`, `slice`, `Promise.then`) use for their results.

13.7 Abstract Class Pattern

```
class Shape {
  constructor() {
    if (new.target === Shape) throw new TypeError("Shape is abstract");
  }
  area() { throw new Error("Not implemented"); }
}
```

13.8 Class Limitations

- No interfaces, no method overloading, no `protected`, no `final`.
- No multiple inheritance (use mixins, §12.7 p. 50).
- Class declarations are hoisted but TDZ'd — you cannot instantiate before the definition.
- Methods are non-enumerable, so `Object.keys(instance)` shows only fields.
- Decorators are a Stage 3 proposal, widely used via TypeScript/Babel.

PART III

Data Structures

14	Arrays	56
15	Destructuring, Spread, and Rest	61
16	Iterators and Iterables	64
17	Generators	66

CHAPTER 14

Arrays

IN THIS CHAPTER

14.1 Creating Arrays 56
14.2 `length` and Sparse Arrays 56
14.3 Mutating Methods 57
14.4 Non-Mutating Methods 57
14.5 Iteration Method Signatures 58
14.6 Sorting in Depth 58
14.7 Copying and Flattening 59
14.8 Common Array Recipes 59
14.9 Array-Likes 60
14.10 Performance Notes 60

Arrays are objects with integer-like string keys and a magic `length` property.

14.1 Creating Arrays

```
[1, 2, 3]
new Array(3)              // [ <3 empty items> ] — length 3, NO elements
new Array(1, 2)           // [1, 2] — inconsistent single-arg behavior
Array.of(3)               // [3] — fixes the above (ES2015)
Array.from("abc")         // ["a","b","c"]
Array.from({length: 3}, (_, i) => i)      // [0, 1, 2]
Array.from(new Set([1,1,2]))              // [1, 2]
Array.from(map)                           // [[k,v], ...]
[...iterable]
new Array(3).fill(0)      // [0, 0, 0]
Array(5).keys()           // iterator 0..4
```

14.2 `length` and Sparse Arrays

`length` is always one more than the highest integer index, and it is **writable.**

```
const a = [1, 2, 3];
a.length = 1;      // a is now [1] — truncates destructively
a.length = 3;      // [1, <2 empty items>] — holes, not undefined

a[10] = 1;         // length becomes 11, indices 1–9 are holes
```

Holes behave inconsistently across methods:

```
const sparse = [1, , 3];
sparse.length;                    // 3
1 in sparse;                      // false
sparse[1];                        // undefined
sparse.forEach(x => log(x));      // skips the hole → 1, 3
sparse.map(x => 0);               // [0, <hole>, 0] — preserves holes
sparse.filter(() => true);        // [1, 3] — removes holes
[...sparse];                      // [1, undefined, 3] — spread fills them
Object.keys(sparse);              // ["0", "2"]
sparse.includes(undefined);       // true
sparse.indexOf(undefined);        // -1
```

Avoid sparse arrays. Use `Array.from({length: n})` or `.fill()` to get dense ones.

14.3 Mutating Methods

These change the array in place:

```
arr.push(...items)         // append, returns new length
arr.pop()                  // remove last, returns it
arr.shift()                // remove first, returns it — O(n)
arr.unshift(...items)      // prepend, returns new length — O(n)
arr.splice(start, deleteCount, ...insert)   // returns removed items
arr.sort(compareFn)        // in place, STABLE since ES2019
arr.reverse()              // in place
arr.fill(value, start, end)
arr.copyWithin(target, start, end)
```

```
arr.splice(2, 0, "x");          // insert at index 2
arr.splice(2, 1);               // remove 1 item at index 2
arr.splice(-1);                 // remove last
```

14.4 Non-Mutating Methods

```
arr.concat(...arraysOrValues)
arr.slice(start, end)              // shallow copy; slice() clones
arr.join(separator)                // null/undefined become ""
arr.indexOf(x, from)               // === , -1 if absent
arr.lastIndexOf(x, from)
arr.includes(x, from)              // SameValueZero — finds NaN
arr.find(fn)                       // first matching VALUE
arr.findIndex(fn)
arr.findLast(fn)                   // ES2023
arr.findLastIndex(fn)              // ES2023
arr.at(i)                          // ES2022 — supports negative indices
arr.flat(depth = 1)                // ES2019; Infinity for full flatten
arr.flatMap(fn)                    // map then flat(1)
arr.filter(fn)
```

```
arr.map(fn)
arr.reduce(fn, init)
arr.reduceRight(fn, init)
arr.some(fn)                        // short-circuits true
arr.every(fn)                       // short-circuits false
arr.forEach(fn)                     // returns undefined, cannot break
arr.keys() / values() / entries()   // iterators
arr.toString() / toLocaleString()
```

Change-by-copy methods (ES2023) — non-mutating versions of the mutators:

```
arr.toSorted(fn)
arr.toReversed()
arr.toSpliced(start, deleteCount, ...items)
arr.with(index, value)              // copy with one index replaced
```

14.5 Iteration Method Signatures

All take `(element, index, array)` and an optional `thisArg`.

```
arr.map((el, i, whole) => el * 2);
arr.reduce((acc, el, i, whole) => acc + el, 0);
```

`reduce` without an initial value uses `arr[0]` as the seed and starts at index 1 — and **throws `TypeError` on an empty array.** Always pass an initial value.

```
// reduce recipes
const sum = ns.reduce((a, b) => a + b, 0);
const byId = items.reduce((m, x) => (m[x.id] = x, m), {});
const flat = nested.reduce((a, b) => a.concat(b), []);
const grouped = items.reduce((m, x) => ((m[x.type] ??= []).push(x), m), {});
const pipeline = fns.reduce((v, f) => f(v), input);
```

14.6 Sorting in Depth

Default sort converts elements to strings — the classic trap:

```
[10, 9, 1].sort();                  // [1, 10, 9]  ← lexicographic!
[10, 9, 1].sort((a, b) => a - b);   // [1, 9, 10]  ascending
[10, 9, 1].sort((a, b) => b - a);   // [10, 9, 1]  descending

strings.sort((a, b) => a.localeCompare(b));         // locale-correct
objects.sort((a, b) => a.age - b.age || a.name.localeCompare(b.name));  // multi-key
```

The comparator must return negative / zero / positive and be *consistent* (a total order); inconsistent comparators give implementation-defined results. Sort is stable as of ES2019, so equal elements keep their relative order.

`undefined` values always sort to the end and are never passed to the comparator. Holes sort after `undefined`.

14.7 Copying and Flattening

```
const shallow = [...arr];
const shallow = arr.slice();
const shallow = Array.from(arr);
const deep = structuredClone(arr);

[[1,[2]],[3]].flat();          // [1, [2], 3]
[[1,[2]],[3]].flat(Infinity);  // [1, 2, 3]
["a b", "c"].flatMap(s => s.split(" "));   // ["a","b","c"]
```

14.8 Common Array Recipes

```
// unique
[...new Set(arr)]
// unique by key
[...new Map(arr.map(o => [o.id, o])).values()]

// range
Array.from({ length: n }, (_, i) => i)
// zip
a.map((x, i) => [x, b[i]])

// chunk
const chunk = (a, n) => Array.from({length: Math.ceil(a.length/n)}, (_, i) =>
a.slice(i*n, i*n+n));

// shuffle (Fisher-Yates)
for (let i = a.length - 1; i > 0; i--) {
  const j = Math.floor(Math.random() * (i + 1));
  [a[i], a[j]] = [a[j], a[i]];
}

// group (ES2024)
Object.groupBy(items, x => x.type);   // plain object
Map.groupBy(items, x => x.type);      // Map, allows any key type

// intersection / difference / union
const inter = a.filter(x => b.includes(x));
const diff  = a.filter(x => !b.includes(x));
const union = [...new Set([...a, ...b])];

// last element
arr.at(-1)
```

14.9 Array-Likes

Objects with `length` and integer keys but no array methods: `arguments`, `NodeList`, `HTMLCollection`, strings.

```
Array.from(arrayLike);
Array.from(arrayLike, mapFn);
[...iterableOnly];                      // only if it has Symbol.iterator
Array.prototype.slice.call(arrayLike);  // legacy
```

`NodeList` is iterable and has `forEach`; `HTMLCollection` is neither.

14.10 Performance Notes

- `push`/`pop` are O(1); `shift`/`unshift`/`splice` are O(n).
- Engines optimize "packed" arrays of uniform type (SMI, double, object). Creating holes or mixing types deoptimizes to dictionary mode.
- `delete arr[i]` creates a hole — use `splice` instead.
- For large numeric data use TypedArrays (§27 p. 94).

CHAPTER 15

Destructuring, Spread, and Rest

IN THIS CHAPTER

15.1 Array Destructuring 61
15.2 Object Destructuring 61
15.3 Destructuring in Parameters 62
15.4 Spread 62
15.5 Rest 62
15.6 Spread vs Rest Summary 63

15.1 Array Destructuring

```
const [a, b] = [1, 2];
const [a, , c] = [1, 2, 3];              // skip with a hole
const [a = 10] = [];                     // default (only for undefined)
const [a, ...rest] = [1, 2, 3];          // rest → [2, 3]
[a, b] = [b, a];                         // swap (needs a leading ; if ASI risk)
const [[x, y], [z]] = [[1, 2], [3]];     // nested
const [first] = "hello";                 // works on any iterable → "h"
const [k, v] = mapEntry;
for (const [i, v] of arr.entries()) {}
```

15.2 Object Destructuring

```
const { a, b } = obj;
const { a: renamed } = obj;
const { a = 1 } = obj;                    // default
const { a: renamed = 1 } = obj;           // rename + default
const { a, ...rest } = obj;               // rest object (ES2018), own enumerable
only
const { a: { b } } = obj;                 // nested
const { "odd key": ok } = obj;            // string key
const { [dynamicKey]: val } = obj;        // computed key
const { length } = "hello";               // 5
const { a } = null;                       // TypeError
const { a } = {} ?? {};                   // guard
```

Assignment (not declaration) requires parentheses, because `{` at statement start is a block:

```
let a;
({ a } = obj);
```

15.3 Destructuring in Parameters

```
function f({ x = 0, y = 0 } = {}, [first] = []) {}
f();                                  // defaults all the way down

const users = [{ name: "a", address: { city: "X" } }];
users.map(({ name, address: { city } }) => `${name} ${city}`);

// Preserving remaining props:
function config({ id, ...options }) {}
```

15.4 Spread

```
// Arrays / iterables
[...arr1, ...arr2]
[...str]                      // splits by CODE POINT, unlike .split("")
[...new Set(x)]
[...map]                      // [[k, v], ...]
Math.max(...numbers)          // spread into arguments
fn(...args)
new Date(...[2024, 0, 1])

// Objects (ES2018) — own enumerable props, including symbols
{ ...obj }
{ ...defaults, ...overrides }
{ ...obj, extra: 1 }
{ extra: 1, ...obj }          // order matters — obj can overwrite `extra`
{ ...null }                   // {} — nullish spreads are ignored in objects
[...null]                     // TypeError — arrays require an iterable
{ ..."ab" }                   // { 0: "a", 1: "b" }
```

Object spread copies values, so getters are **invoked** and the results become plain data properties. It does not copy the prototype.

15.5 Rest

Collects "the remaining" — must be the **last** element, and cannot have a default.

```
function f(a, ...rest) {}       // rest params → real Array
const [a, ...rest] = arr;
const { a, ...rest } = obj;
```

15.6 Spread vs Rest Summary

Position	Meaning
Function call arguments	spread
Array/object literal	spread
Function parameter list	rest
Destructuring pattern	rest

CHAPTER 16

Iterators and Iterables

IN THIS CHAPTER

16.1 The Two Protocols 64
16.2 What Consumes Iterables 64
16.3 Built-in Iterables 65
16.4 Manual Iteration and Early Exit 65
16.5 Iterator Helpers (ES2025) 65

16.1 The Two Protocols

Iterable protocol: an object with a `[Symbol.iterator]()` method returning an iterator.

Iterator protocol: an object with a `next()` method returning `{ value, done }`. Optionally `return(value)` (called on early exit) and `throw(err)`.

```
const iterable = {
  [Symbol.iterator]() {
    let i = 0;
    return {
      next: () => i < 3 ? { value: i++, done: false } : { value: undefined, done:
true },
      return(v) { /* cleanup on break/throw */ return { value: v, done: true }; },
      [Symbol.iterator]() { return this; },   // makes the ITERATOR itself iterable
    };
  },
};
[...iterable];   // [0, 1, 2]
```

16.2 What Consumes Iterables

```
for (const x of iterable) {}
[...iterable]
Array.from(iterable)
const [a, b] = iterable
new Set(iterable) / new Map(iterable) / new WeakSet(...)
Promise.all(iterable) / race / allSettled / any
yield* iterable
Object.fromEntries(iterable)
fn(...iterable)
```

16.3 Built-in Iterables

`String`, `Array`, `TypedArray`, `Map`, `Set`, `arguments`, `NodeList`, generators, and (async) streams.

Plain objects are **not** iterable — use `Object.entries(obj)`.

```
// Make a plain object iterable:
obj[Symbol.iterator] = function* () { yield* Object.values(this); };
```

16.4 Manual Iteration and Early Exit

```
const it = arr[Symbol.iterator]();
it.next();   // { value: ..., done: false }

// `break` in for...of calls it.return() so resources can be released
for (const line of fileLines()) { if (line === "") break; }
```

16.5 Iterator Helpers (ES2025)

Lazy, chainable methods on `Iterator.prototype` — they avoid materializing intermediate arrays.

```
iterator.map(fn)
iterator.filter(fn)
iterator.take(n)
iterator.drop(n)
iterator.flatMap(fn)
iterator.reduce(fn, init)
iterator.toArray()
iterator.forEach(fn)
iterator.some(fn) / every(fn) / find(fn)
Iterator.from(iterableOrIterator)

// Infinite sequence, lazily consumed:
function* naturals() { let n = 0; while (true) yield n++; }
naturals().filter(n => n % 3 === 0).take(4).toArray();   // [0, 3, 6, 9]
```

CHAPTER 17

Generators

IN THIS CHAPTER

17.1 Syntax and Basics 66
17.2 Two-Way Communication 66
17.3 `return()` and `throw()` 67
17.4 Delegation with `yield*` 67
17.5 Practical Uses 67
17.6 Async Generators 68

A generator function returns a **generator object** that is both an iterator and an iterable, and whose execution can be paused and resumed.

17.1 Syntax and Basics

```
function* gen() {
  yield 1;
  yield 2;
  return 3;          // becomes { value: 3, done: true }, skipped by for...of/spread
}

const g = gen();
g.next();   // { value: 1, done: false }
g.next();   // { value: 2, done: false }
g.next();   // { value: 3, done: true }
g.next();   // { value: undefined, done: true }

[...gen()];             // [1, 2] — return value excluded
for (const v of gen()) {}   // 1, 2
```

Forms: `function* g(){}`, `const g = function* (){}`, `*method(){}` in objects/classes, `static *m(){}`, `async function* (){}`. **Arrow functions cannot be generators.**

17.2 Two-Way Communication

`yield` is an expression; the value passed to `next(v)` becomes its result.

```
function* dialog() {
  const name = yield "What's your name?";
  const age = yield `Hi ${name}, age?`;
  return `${name} is ${age}`;
}

const d = dialog();
d.next();          // { value: "What's your name?", done: false }
d.next("Ada");     // { value: "Hi Ada, age?", done: false }
d.next(36);        // { value: "Ada is 36", done: true }
```

The **first** `next()` argument is discarded — there is no `yield` waiting for it yet.

17.3 `return()` and `throw()`

```
g.return(99);   // forces completion → { value: 99, done: true }, runs finally blocks
g.throw(err);   // throws AT the paused yield; the generator can catch it
```

```
function* resource() {
  try { yield open(); }
  finally { close(); }     // runs on break, return(), or throw()
}
```

17.4 Delegation with `yield*`

```
function* inner() { yield 1; yield 2; return "inner done"; }
function* outer() {
  const r = yield* inner();     // yields 1, 2; r === "inner done"
  yield 3;
}
[...outer()];   // [1, 2, 3]
```

`yield*` works on any iterable and forwards `next`/`throw`/`return` through to the delegate.

17.5 Practical Uses

```
// Infinite / lazy sequences
function* fib() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a +
b]; } }

// ID generation
function* ids(start = 0) { while (true) yield start++; }

// Tree traversal without building an array
function* walk(node) {
  yield node;
```

```
  for (const child of node.children ?? []) yield* walk(child);
}

// Chunking a stream
function* chunks(iter, size) {
  let buf = [];
  for (const x of iter) { buf.push(x); if (buf.length === size) { yield buf; buf = []
; } }
  if (buf.length) yield buf;
}

// Custom iterables, one line
class Range {
  constructor(a, b) { this.a = a; this.b = b; }
  *[Symbol.iterator]() { for (let i = this.a; i < this.b; i++) yield i; }
}
```

Historically, generators powered coroutine-based async (co, redux-saga) before `async/await` existed; `async/await` is essentially a generator driven by a promise-resolving runner.

17.6 Async Generators

```
async function* pages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next);
    const data = await res.json();
    yield* data.items;
    next = data.next;
  }
}

for await (const item of pages("/api")) { process(item); }
```

Async generators implement `Symbol.asyncIterator` and their `next()` returns a Promise.

PART IV

Standard Library

18	String	70
19	Number and Math	74
20	BigInt	77
21	Date and Temporal	78
22	Regular Expressions	81
23	JSON	85
24	Map, Set, WeakMap, WeakSet, WeakRef	87
25	Symbol	90
26	Proxy and Reflect	92
27	Typed Arrays, ArrayBuffer, DataView	94
28	Intl — Internationalization	97

CHAPTER 18

String

IN THIS CHAPTER

18.1 Creating Strings 70
18.2 UTF-16, Code Units, and Code Points 70
18.3 Template Literals 71
18.4 Complete Method Reference 71
18.5 `replace` in Depth 72
18.6 Comparison and Sorting 73
18.7 String Immutability and Performance 73

18.1 Creating Strings

```
'single'   "double"   `template`
String(x)                          // conversion
x.toString()
`multi
line`                              // template literals preserve newlines
'\n \t \\ \' \" \0 \xA9 \u00A9 \u{1F600}'
```

Escape sequences: `\n` LF, `\r` CR, `\t` tab, `\b` backspace, `\f` form feed, `\v` vertical tab, `\0` NUL, `\\`, `\'`, `\"`, `\`` , `\xHH` (2 hex digits), `\uHHHH` (4), `\u{HHHHH}` (code point, ES2015).

18.2 UTF-16, Code Units, and Code Points

Strings are sequences of 16-bit code units. Characters above U+FFFF are stored as **surrogate pairs** of two units.

```
const emoji = "😀";
emoji.length;              // 2   ← code units, not characters
emoji[0];                  // lone high surrogate, renders as garbage
[...emoji].length;         // 1   ← spread iterates by CODE POINT
Array.from(emoji).length;  // 1
emoji.codePointAt(0);      // 128512
emoji.charCodeAt(0);       // 55357 (high surrogate)
String.fromCodePoint(128512);  // "😀"
"e\u0301".length;          // 2 — "é" as e + combining accent
"e\u0301".normalize("NFC").length;   // 1

// Grapheme clusters (user-perceived characters) need Intl:
```

```
const seg = new Intl.Segmenter("en", { granularity: "grapheme" });
[...seg.segment(" 🤦")].length;   // 1
```

Rule of thumb: `.length` for storage, spread/`Intl.Segmenter` for anything user-facing.

18.3 Template Literals

```
`Hello ${name}`
`${a + b} and ${obj.method()}`
`nested ${`inner ${x}`}`
tag`text ${v}`                        // tagged template (§5.13)
String.raw`C:\dir\n`                  // raw, no escape processing
```

Template literals coerce with `ToString`, so `${obj}` calls `toString()` and `${null}` gives `"null"`.

18.4 Complete Method Reference

Inspection

```
str.length
str.at(i)               // ES2022, negative OK
str.charAt(i)
str[i]
str.charCodeAt(i)       // UTF-16 unit
str.codePointAt(i)      // full code point
```

Searching

```
str.indexOf(sub, from)
str.lastIndexOf(sub, from)
str.includes(sub, from)
str.startsWith(sub, from)
str.endsWith(sub, end)
str.search(regexp)              // index or -1
str.match(regexp)               // array or null; with /g → all matches, no groups
str.matchAll(regexp)            // ES2020, iterator of full match objects, needs /g
```

Extraction

```
str.slice(start, end)      // negatives count from the end — use this one
str.substring(start, end)  // negatives → 0; swaps args if start > end
str.substr(start, length)  // deprecated (Annex B)
str.split(sep, limit)      // sep can be a string or regex; "" splits by code UNIT
```

Transformation

```
str.toUpperCase() / toLowerCase()
str.toLocaleUpperCase(locale) / toLocaleLowerCase(locale)   // handles Turkish i,
etc.
str.trim() / trimStart() / trimEnd()
str.padStart(len, pad) / padEnd(len, pad)     // ES2017
str.repeat(n)
str.concat(...strs)                            // prefer + or template
str.replace(pattern, replacement)              // first match only for strings
str.replaceAll(pattern, replacement)           // ES2021; regex must be /g
str.normalize("NFC" | "NFD" | "NFKC" | "NFKD")
str.localeCompare(other, locales, options)
```

Statics

```
String.fromCharCode(...units)
String.fromCodePoint(...points)
String.raw(strings, ...values)
```

18.5 `replace` in Depth

Replacement string patterns:

Pattern	Meaning
`$$`	literal `$`
`$&`	the whole match
`$``	text before the match
`$'`	text after the match
`$1`–`$99`	numbered capture group
`$<name>`	named capture group

```
"2024-01-15".replace(/(\d+)-(\d+)-(\d+)/, "$3/$2/$1");   // "15/01/2024"
"a-b".replace(/(?<x>\w)-(?<y>\w)/, "$<y>-$<x>");         // "b-a"

// Function replacement — full control
str.replace(/\d+/g, (match, p1, offset, whole, groups) => Number(match) * 2);
"hello".replace(/l/g, (m, off) => off);   // "he23o"

// Escaping a string for use in a regex:
const esc = s => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
```

18.6 Comparison and Sorting

```
"a" < "b";                       // code-unit order — "Z" < "a", "é" > "z"
"é".localeCompare("z", "en");    // -1 — locale-correct
new Intl.Collator("de", { sensitivity: "base" }).compare("a", "ä");  // 0
arr.sort(new Intl.Collator("en").compare);   // fastest for repeated comparisons
```

18.7 String Immutability and Performance

Strings cannot be mutated in place. Repeated concatenation in a loop is fine in modern engines (they use ropes/cons-strings), but for very large builds collect into an array and `join("")`.

CHAPTER 19

Number and Math

IN THIS CHAPTER

19.1 Number Statics 74
19.2 Number Instance Methods 74
19.3 Math Reference 75
19.4 Floating-Point Strategy 75

19.1 Number Statics

```
Number.MAX_SAFE_INTEGER      // 2^53 - 1
Number.MIN_SAFE_INTEGER
Number.MAX_VALUE
Number.MIN_VALUE             // smallest positive
Number.EPSILON
Number.POSITIVE_INFINITY / NEGATIVE_INFINITY
Number.NaN

Number.isInteger(v)
Number.isSafeInteger(v)
Number.isFinite(v)           // no coercion (global isFinite coerces)
Number.isNaN(v)              // no coercion (global isNaN coerces)
Number.parseFloat / parseInt // identical to the globals
```

19.2 Number Instance Methods

```
(1234.5678).toFixed(2);          // "1234.57"  ← STRING, and rounds oddly
(0.615).toFixed(2);              // "0.61" not "0.62" — binary representation
(1234.5678).toPrecision(6);      // "1234.57"
(255).toString(2);               // "11111111"  radix 2–36
(255).toString(16);              // "ff"
parseInt("ff", 16);              // 255
(1234.5678).toExponential(2);    // "1.23e+3"
(1234567.891).toLocaleString("en-US");  // "1,234,567.891"
(1234.5).toLocaleString("en-US", { style: "currency", currency: "USD" });  //
"$1,234.50"
```

Note `1..toString()` or `(1).toString()` — a single dot after an integer literal is parsed as a decimal point.

19.3 Math Reference

Constants

```
Math.PI       // 3.14159...
Math.E        // 2.71828...
Math.LN2, Math.LN10, Math.LOG2E, Math.LOG10E, Math.SQRT2, Math.SQRT1_2
```

Rounding

```
Math.round(x)     // half away from zero for positives: 2.5→3, -2.5→-2
Math.floor(x)     // toward -Infinity
Math.ceil(x)      // toward +Infinity
Math.trunc(x)     // toward zero (ES2015)
Math.fround(x)    // nearest float32
```

Arithmetic

```
Math.abs, Math.sign, Math.sqrt, Math.cbrt, Math.pow(x,y)
Math.hypot(...ns)          // sqrt of sum of squares, overflow-safe
Math.exp, Math.expm1, Math.log, Math.log1p, Math.log2, Math.log10
Math.min(...ns)            // Infinity with no args
Math.max(...ns)            // -Infinity with no args
Math.imul(a, b)            // 32-bit integer multiply
Math.clz32(x)              // count leading zeros
Math.random()              // [0, 1) — NOT cryptographically secure
```

Trigonometry

```
Math.sin, cos, tan, asin, acos, atan, atan2(y, x)
Math.sinh, cosh, tanh, asinh, acosh, atanh
```

Recipes

```
const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
const clamp = (v, lo, hi) => Math.min(Math.max(v, lo), hi);
const round = (v, dp) => Math.round(v * 10**dp) / 10**dp;
const deg = r => r * 180 / Math.PI;

// Cryptographically secure randomness:
crypto.getRandomValues(new Uint32Array(1))[0];
crypto.randomUUID();
```

19.4 Floating-Point Strategy

For money and exact decimals, never use floats directly:

- Work in the smallest integer unit (cents) with `Number` (safe below 2^53) or `BigInt`.

- Or use a decimal library (decimal.js, big.js, dinero.js).
- Format only at the boundary with `Intl.NumberFormat`.

CHAPTER 20

BigInt

Arbitrary-precision integers (ES2020).

```
9007199254740993n
BigInt(9007199254740993);        // ⚠️  imprecise — the Number was already wrong
BigInt("9007199254740993");      // ✅  exact
BigInt.asIntN(64, x);            // wrap to signed 64-bit
BigInt.asUintN(64, x);
```

Rules:

```
10n + 20n;        // 30n
10n / 3n;         // 3n    ← truncating integer division
10n ** 100n;      // huge, exact
// 10n + 10;      ← TypeError: cannot mix BigInt and other types
10n == 10;        // true   (loose equality compares mathematical values)
10n === 10;       // false  (different types)
10n > 9;          // true   (relational comparison IS allowed across types)
typeof 10n;       // "bigint"
Number(10n);      // 10 — may lose precision
// Math.max(1n);  ← TypeError: Math doesn't accept BigInt
// JSON.stringify(1n)  ← TypeError; add a toJSON or replacer
// +10n           ← TypeError: unary plus is not supported
-10n;             // fine
Boolean(0n);      // false
```

Use for: cryptography, large IDs (Twitter snowflakes, DB bigints), precise financial integers, high-precision math.

CHAPTER 21

Date and Temporal

IN THIS CHAPTER

21.1 The `Date` Object 78
21.2 Getters and Setters 78
21.3 Formatting 79
21.4 Arithmetic 79
21.5 Known Date Problems 79
21.6 Temporal (Stage 3, shipping) 79

21.1 The `Date` Object

`Date` stores milliseconds since the Unix epoch (1970-01-01T00:00:00Z) internally, but nearly all its accessors are local-timezone.

```
new Date();                       // now
new Date(1700000000000);          // from epoch ms
new Date("2024-01-15");           // ⚠ parsed as UTC (date-only ISO)
new Date("2024-01-15T10:00:00");  // ⚠ parsed as LOCAL (with time, no Z)
new Date("January 15, 2024");     // implementation-dependent — avoid
new Date(2024, 0, 15, 10, 30, 0, 0);   // ⚠ MONTH IS 0-INDEXED, local time
Date.now();                       // epoch ms, no object allocated
Date.UTC(2024, 0, 15);            // epoch ms from UTC parts
new Date(2024, 0, 32);            // rolls over to Feb 1 — no error
```

21.2 Getters and Setters

```
// Local time
getFullYear() getMonth() getDate() getDay() getHours() getMinutes()
getSeconds() getMilliseconds() getTimezoneOffset()   // minutes, INVERTED sign

// UTC equivalents
getUTCFullYear() getUTCMonth() ...

// Setters mirror the getters
setFullYear(y, m, d) setMonth(m) setDate(d) setHours(h, m, s, ms) ...

getTime()  valueOf()   // epoch ms
```

`getMonth()` returns 0–11. `getDay()` returns 0 (Sunday) – 6. `getDate()` is the day of the month. `getYear()` is deprecated.

21.3 Formatting

```
d.toISOString();          // "2024-01-15T10:00:00.000Z" — always UTC, the safe one
d.toJSON();               // same as toISOString
d.toString();             // implementation/locale-dependent, avoid for storage
d.toDateString();
d.toLocaleDateString("en-GB", { dateStyle: "long", timeZone: "Asia/Kolkata" });
d.toLocaleString("en-US", { timeZone: "UTC", hour12: false });

new Intl.DateTimeFormat("en-IN", {
  dateStyle: "full", timeStyle: "short", timeZone: "Asia/Kolkata",
}).format(d);
```

21.4 Arithmetic

```
const diffMs = d2 - d1;                 // dates coerce to numbers
const days = diffMs / 86400000;

const tomorrow = new Date(d);
tomorrow.setDate(d.getDate() + 1);      // handles month/year rollover correctly

// Relative formatting:
new Intl.RelativeTimeFormat("en", { numeric: "auto" }).format(-1, "day");  //
"yesterday"
```

Date is mutable — `setDate` modifies in place. Always copy first: `new Date(d)`.

21.5 Known Date Problems

- Month is 0-indexed; day-of-month is not.
- String parsing is only reliable for full ISO 8601; everything else is implementation-defined.
- Date-only ISO strings parse as UTC, date-time strings without `Z` parse as local — a constant source of off-by-one-day bugs.
- No timezone support beyond the system zone and UTC for *construction*.
- Mutable API.
- DST transitions make "add 1 day" ambiguous.

21.6 Temporal (Stage 3, shipping)

The replacement API — immutable, explicit about timezones and calendars.

```
Temporal.Now.plainDateISO();                  // 2026-08-24
Temporal.Now.zonedDateTimeISO("Asia/Kolkata");
Temporal.PlainDate.from("2024-01-15").add({ days: 1 });     // returns a NEW date
Temporal.Duration.from({ hours: 2 });
Temporal.Instant.from("2024-01-15T10:00Z");
zdt.withTimeZone("America/New_York");
d1.until(d2, { largestUnit: "day" });
```

Types: `Instant`, `ZonedDateTime`, `PlainDate`, `PlainTime`, `PlainDateTime`, `PlainYearMonth`, `PlainMonthDay`, `Duration`, `TimeZone`, `Calendar`. Until it ships everywhere, use a polyfill or a library (date-fns, Luxon, Day.js).

CHAPTER 22

Regular Expressions

IN THIS CHAPTER

22.1 Creating 81
22.2 Flags 81
22.3 Pattern Syntax 81
22.4 Methods 83
22.5 The Match Object 83
22.6 `lastIndex` Statefulness 83
22.7 Catastrophic Backtracking (ReDoS) 83
22.8 Useful Patterns 84

22.1 Creating

```
/pattern/flags                          // literal, compiled once
new RegExp("pattern", "flags")          // dynamic — remember to double-escape
new RegExp(`^${escapeRegExp(user)}$`);
```

22.2 Flags

Flag	Name	Effect
`g`	global	find all matches; enables `lastIndex`
`i`	ignoreCase	case-insensitive
`m`	multiline	`^` / `$` match at line breaks
`s`	dotAll (ES2018)	`.` matches newlines
`u`	unicode (ES2015)	code-point mode, enables `\u{...}` and `\p{...}`
`v`	unicodeSets (ES2024)	superset of `u`, adds set operations in classes
`y`	sticky (ES2015)	match must start exactly at `lastIndex`
`d`	hasIndices (ES2022)	adds `.indices` with match offsets

22.3 Pattern Syntax

Character classes

```
.        any char except line terminators (unless /s)
\d \D    digit / non-digit
\w \W    [A-Za-z0-9_] / negation
\s \S    whitespace / non-whitespace
[abc]    any of
[^abc]   none of
[a-z]    range
\p{L} \p{Lu} \p{Script=Greek} \p{Emoji}    Unicode property (requires /u or /v)
\P{...}  negated Unicode property
```

Anchors and boundaries

```
^  $     start / end (of string, or line with /m)
\b \B    word boundary / non-boundary
```

Quantifiers (greedy by default; add `?` for lazy)

```
*        0 or more        *?
+        1 or more         +?
?        0 or 1            ??
{n}      exactly n
{n,}     n or more         {n,}?
{n,m}    between n and m   {n,m}?
```

Groups and references

```
(x)              capturing group
(?:x)            non-capturing group
(?<name>x)       named group (ES2018)
\1               backreference to group 1
\k<name>         named backreference
x|y              alternation
```

Lookaround (ES2018 added lookbehind)

```
(?=x)     positive lookahead
(?!x)     negative lookahead
(?<=x)    positive lookbehind
(?<!x)    negative lookbehind
```

Escapes

```
\n \r \t \f \v \0
\xHH  \uHHHH  \u{HHHHH}  (last needs /u)
\\ \. \* \+ \? \( \) \[ \] \{ \} \| \^ \$ \/
```

22.4 Methods

```
re.test(str);              // boolean — ⚠ stateful with /g or /y
re.exec(str);              // match array or null; with /g, call repeatedly
str.match(re);             // no /g → match object; with /g → array of strings only
str.matchAll(re);          // iterator of full match objects (needs /g)
str.search(re);            // index or -1
str.replace(re, repl);
str.replaceAll(re, repl);  // regex must have /g
str.split(re);             // capture groups are included in the output
```

22.5 The Match Object

```
const m = /(?<y>\d{4})-(?<m>\d{2})/d.exec("on 2024-01-15");
m[0];          // "2024-01"    full match
m[1], m[2];    // "2024", "01" numbered groups
m.groups.y;    // "2024"       named groups
m.index;       // 3
m.input;       // "on 2024-01-15"
m.indices;     // [[3,10],[3,7],[8,10]]  with the /d flag
```

22.6 `lastIndex` Statefulness

A regex with `g` or `y` remembers where it stopped — this is the #1 regex bug in JS.

```
const re = /a/g;
re.test("a");   // true
re.test("a");   // false ← lastIndex is now 1
re.lastIndex = 0;   // manual reset

// Safer: create the regex fresh, use matchAll, or omit /g for test()
for (const m of str.matchAll(/a/g)) {}   // matchAll clones internally
```

Also: never share a `/g` regex across modules or reuse one in a loop without resetting.

22.7 Catastrophic Backtracking (ReDoS)

Nested quantifiers over overlapping alternatives can take exponential time.

```
/(a+)+$/.test("aaaaaaaaaaaaaaaaaaaaaaaaaaX");   // hangs
```

Mitigate: avoid nesting quantifiers, make inner patterns mutually exclusive, use atomic-style rewrites, cap input length, or run untrusted patterns in a worker with a timeout.

22.8 Useful Patterns

```
const email    = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;         // pragmatic, not RFC-complete
const uuid     = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-
f]{12}$/i;
const isoDate  = /^\d{4}-\d{2}-\d{2}$/;
const hexColor = /^#(?:[0-9a-f]{3}|[0-9a-f]{6})$/i;
const slug     = /^[a-z0-9]+(?:-[a-z0-9]+)*$/;
const camelToKebab = s => s.replace(/([a-z0-9])([A-Z])/g, "$1-$2").toLowerCase();
const escapeRegExp = s => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
```

Validate emails by sending mail, not by regex. Parse HTML with a parser, not a regex.

CHAPTER 23

JSON

IN THIS CHAPTER

23.1 JSON.stringify 85
23.2 JSON.parse 86
23.3 JSON Format Rules 86
23.4 Deep Clone Caveats 86

23.1 JSON.stringify

```
JSON.stringify(value, replacer?, space?)

JSON.stringify({a: 1});                  // '{"a":1}'
JSON.stringify(obj, null, 2);            // pretty-printed, 2-space indent
JSON.stringify(obj, ["a", "b"]);         // allowlist of keys
JSON.stringify(obj, (k, v) => typeof v === "bigint" ? v.toString() : v);
```

What gets dropped or transformed:

```
JSON.stringify({ u: undefined, f() {}, s: Symbol() });  // "{}"  — all omitted
JSON.stringify([undefined, function(){}, Symbol()]);    // "[null,null,null]"
JSON.stringify(NaN);              // "null"
JSON.stringify(Infinity);         // "null"
JSON.stringify(new Date());       // ISO string via toJSON()
JSON.stringify(new Map([[1,2]])); // "{}"  — Maps/Sets lose everything
JSON.stringify(1n);               // TypeError
JSON.stringify(circular);         // TypeError: Converting circular structure
```

Only **own enumerable string-keyed** properties are serialized. A `toJSON()` method, if present, is called first and its result serialized instead.

```
class Money {
  constructor(c) { this.cents = c; }
  toJSON() { return { amount: this.cents / 100 }; }
}
```

`JSON.stringify` is well-formed since ES2019 — lone surrogates are escaped rather than emitted raw.

23.2 `JSON.parse`

```
JSON.parse(text, reviver?)

JSON.parse('{"d":"2024-01-15T00:00:00Z"}', (k, v) =>
  typeof v === "string" && /^\d{4}-\d{2}-\d{2}T/.test(v) ? new Date(v) : v
);
```

The reviver visits every key bottom-up; returning `undefined` deletes the key. As of ES2025, the reviver's third argument gives `{ source }`, the raw text — useful for preserving big-number precision.

`JSON.parse` throws `SyntaxError` on invalid input — always wrap it.

23.3 JSON Format Rules

- Keys must be double-quoted strings.
- No trailing commas, no comments, no single quotes.
- Values: object, array, string, number, `true`, `false`, `null`.
- No `undefined`, `NaN`, `Infinity`, functions, dates, or comments.
- Numbers have no precision guarantee beyond IEEE-754 in practice.

23.4 Deep Clone Caveats

`JSON.parse(JSON.stringify(x))` is a common but lossy clone: it drops `undefined`, functions, and symbols; converts `Date` to string; turns `NaN`/`Infinity` into `null`; loses `Map`/`Set`/`RegExp`; and throws on cycles. Prefer `structuredClone(x)`.

CHAPTER 24

Map, Set, WeakMap, WeakSet, WeakRef

IN THIS CHAPTER

24.1 Map 87
24.2 Set 88
24.3 WeakMap and WeakSet 88
24.4 WeakRef and FinalizationRegistry (ES2021) 89

24.1 Map

Keyed collection where keys may be **any value**, with guaranteed insertion order.

```
const m = new Map([["a", 1], ["b", 2]]);

m.set(key, value);       // chainable
m.get(key);              // undefined if absent
m.has(key);
m.delete(key);           // boolean
m.clear();
m.size;                  // property, not a method

m.keys(); m.values(); m.entries();
m.forEach((value, key, map) => {});
for (const [k, v] of m) {}
[...m];                              // [[k, v], ...]
Object.fromEntries(m);               // → object (string keys only)
new Map(Object.entries(obj));        // ← from object
Map.groupBy(items, fn);              // ES2024
```

Key equality is **SameValueZero**: `NaN` works as a key; `+0` and `-0` are the same key; objects compare by reference.

Map vs Object:

	Map	Object
Key types	any	string / symbol
Order	insertion	integer keys first, then insertion

	Map	Object
Size	`.size`	`Object.keys(o).length`
Iterable	yes	no
Prototype collisions	none	`__proto__`, `toString`, etc.
Frequent add/delete	optimized	slower
JSON	not serializable	native

Use `Map` for dynamic keyed data; use objects for records with known shape.

24.2 Set

Unique values, insertion-ordered.

```
const s = new Set([1, 2, 2, 3]);   // {1, 2, 3}
s.add(v); s.has(v); s.delete(v); s.clear(); s.size;
[...s];                            // array
s.forEach(v => {});
```

Set methods (ES2025):

```
a.union(b)
a.intersection(b)
a.difference(b)
a.symmetricDifference(b)
a.isSubsetOf(b)
a.isSupersetOf(b)
a.isDisjointFrom(b)
```

`has` is O(1) versus `Array.includes` at O(n) — a major win for membership testing in loops.

24.3 WeakMap and WeakSet

Keys must be objects (or, since ES2023, non-registered symbols) and are held **weakly** — an entry does not prevent garbage collection of its key.

```
const wm = new WeakMap();
wm.set(obj, metadata);
wm.get(obj); wm.has(obj); wm.delete(obj);
// NO size, NO iteration, NO clear — by design (GC timing must not be observable)
```

Uses: private per-object data, caches keyed by object, DOM node metadata, memoization tied to object lifetime.

```
const privates = new WeakMap();
class Counter {
  constructor() { privates.set(this, { n: 0 }); }
  inc() { return ++privates.get(this).n; }
}
```

24.4 WeakRef and FinalizationRegistry (ES2021)

```
const ref = new WeakRef(obj);
const maybe = ref.deref();     // the object, or undefined if collected

const reg = new FinalizationRegistry(heldValue => cleanup(heldValue));
reg.register(obj, "id-123", unregisterToken);
reg.unregister(unregisterToken);
```

Cleanup callbacks are **not guaranteed** to run, may run late, and may never run at program exit. Treat them as a best-effort optimization, never as a correctness mechanism. Prefer explicit `dispose()` / `try...finally`.

CHAPTER 25

Symbol

IN THIS CHAPTER

25.1 Creating and Using 90
25.2 The Global Symbol Registry 90
25.3 Well-Known Symbols 90
25.4 Symbols as Property Keys 91

25.1 Creating and Using

```
const s = Symbol();
const s = Symbol("description");     // description is for debugging only
s.description;                       // "description" (ES2019)
s.toString();                        // "Symbol(description)"
typeof s;                            // "symbol"
Symbol("a") === Symbol("a");         // false — always unique
// `${s}`                            ← TypeError: implicit string conversion
String(s);                           // ✅  "Symbol(a)" — explicit is allowed
```

25.2 The Global Symbol Registry

```
Symbol.for("app.key");                    // creates or retrieves — shared across
realms
Symbol.for("app.key") === Symbol.for("app.key");   // true
Symbol.keyFor(Symbol.for("x"));           // "x"
Symbol.keyFor(Symbol("x"));               // undefined
```

25.3 Well-Known Symbols

These are the language's extension hooks.

Symbol	Customizes
`Symbol.iterator`	`for...of`, spread, destructuring
`Symbol.asyncIterator`	`for await...of`
`Symbol.hasInstance`	`instanceof`
`Symbol.toPrimitive`	object → primitive conversion

Symbol	Customizes
`Symbol.toStringTag`	`Object.prototype.toString` result
`Symbol.species`	constructor used by derived methods
`Symbol.isConcatSpreadable`	`Array.prototype.concat` behavior
`Symbol.match` / `matchAll` / `replace` / `search` / `split`	the corresponding `String` methods
`Symbol.unscopables`	which properties `with` ignores

```
class Even {
  static [Symbol.hasInstance](n) { return n % 2 === 0; }
}
4 instanceof Even;   // true

class Collection {
  get [Symbol.toStringTag]() { return "Collection"; }
}
Object.prototype.toString.call(new Collection());   // "[object Collection]"
```

25.4 Symbols as Property Keys

```
const HIDDEN = Symbol("hidden");
obj[HIDDEN] = 1;

Object.keys(obj);                       // symbol not included
JSON.stringify(obj);                    // symbol not included
for (const k in obj) {}                 // not included
Object.getOwnPropertySymbols(obj);      // [HIDDEN]
Reflect.ownKeys(obj);                   // includes it
{ ...obj }                              // DOES copy symbol keys
```

Symbols give non-colliding, non-enumerable-by-default keys — useful for library metadata on user objects. They are *not* a security mechanism (they're discoverable via reflection); use `#private` for real privacy.

CHAPTER 26

Proxy and Reflect

26.1 Proxy

Wraps a target object and intercepts fundamental operations.

```
const proxy = new Proxy(target, handler);
```

All 13 traps:

```
const handler = {
  get(target, prop, receiver) {},
  set(target, prop, value, receiver) {},           // must return true/false
  has(target, prop) {},                            // `in`
  deleteProperty(target, prop) {},                 // `delete`
  ownKeys(target) {},                              // Object.keys, for...in, etc.
  getOwnPropertyDescriptor(target, prop) {},
  defineProperty(target, prop, descriptor) {},
  getPrototypeOf(target) {},
  setPrototypeOf(target, proto) {},
  isExtensible(target) {},
  preventExtensions(target) {},
  apply(target, thisArg, args) {},                 // function call
  construct(target, args, newTarget) {},           // `new`
};
```

```
// Default values
const withDefault = (obj, d) => new Proxy(obj, {
  get: (t, p) => (p in t ? t[p] : d),
});

// Validation
const validated = new Proxy({}, {
  set(t, p, v) {
    if (p === "age" && !Number.isInteger(v)) throw new TypeError("age must be an
integer");
    t[p] = v;
    return true;
  },
});

// Negative array indices
const negIndex = arr => new Proxy(arr, {
```

```
  get: (t, p) => typeof p === "string" && /^-\d+$/.test(p) ? t[t.length + +p] :
t[p], });

// Revocable
const { proxy, revoke } = Proxy.revocable(target, handler);
revoke();   // any further operation throws TypeError
```

Invariants: traps cannot lie about non-configurable/non-writable properties or about extensibility — violating this throws `TypeError`. Proxies also cannot intercept `#private` fields, and internal-slot objects (`Map`, `Date`, `Promise`) need their methods bound to the target.

Proxies are used by Vue 3's reactivity, immer, mock libraries, and observable/ORM layers. They are meaningfully slower than direct property access.

26.2 Reflect

A namespace of the same fundamental operations as functions — the natural companion to Proxy.

```
Reflect.get(target, prop, receiver?)
Reflect.set(target, prop, value, receiver?)      // returns boolean
Reflect.has(target, prop)
Reflect.deleteProperty(target, prop)
Reflect.ownKeys(target)                          // strings + symbols, all own
Reflect.getOwnPropertyDescriptor(target, prop)
Reflect.defineProperty(target, prop, desc)       // returns boolean, no throw
Reflect.getPrototypeOf(target)
Reflect.setPrototypeOf(target, proto)
Reflect.isExtensible(target)
Reflect.preventExtensions(target)
Reflect.apply(fn, thisArg, args)
Reflect.construct(Ctor, args, newTarget?)
```

Advantages over the `Object.*` equivalents: they return booleans instead of throwing, they take a `receiver` so getters/setters resolve `this` correctly through proxies, and `Reflect.construct` supports `new.target`.

```
// Idiomatic pass-through proxy:
new Proxy(target, {
  get(t, p, r) { log(p); return Reflect.get(t, p, r); },
  set(t, p, v, r) { return Reflect.set(t, p, v, r); },
});
```

CHAPTER 27

Typed Arrays, ArrayBuffer, DataView

IN THIS CHAPTER

27.1 ArrayBuffer 94
27.2 The TypedArray Types 94
27.3 DataView 95
27.4 Base64 and Text Conversion 95
27.5 Atomics 96

For binary data: files, network protocols, WebGL, audio, WASM.

27.1 ArrayBuffer

A fixed-length raw byte buffer. You cannot read it directly — you need a view.

```
const buf = new ArrayBuffer(16);      // 16 bytes, zero-filled
buf.byteLength;                        // 16
buf.slice(0, 8);                       // copy
new ArrayBuffer(8, { maxByteLength: 64 });   // resizable (ES2024)
buf.resize(32); buf.transfer();
```

`SharedArrayBuffer` shares memory across workers and is used with `Atomics`; it requires cross-origin isolation headers in browsers.

27.2 The TypedArray Types

Type	Bytes	Range
`Int8Array`	1	−128…127
`Uint8Array`	1	0…255
`Uint8ClampedArray`	1	0…255, clamps instead of wrapping
`Int16Array`	2	−32768…32767
`Uint16Array`	2	0…65535
`Int32Array`	4	-2^{31}…$2^{31}-1$
`Uint32Array`	4	0…$2^{32}-1$
`Float32Array`	4	32-bit float

Type	Bytes	Range
`Float64Array`	8	64-bit float
`BigInt64Array`	8	64-bit signed BigInt
`BigUint64Array`	8	64-bit unsigned BigInt

```
const a = new Int32Array(4);              // own buffer, 16 bytes
const b = new Int32Array([1, 2, 3]);
const c = new Int32Array(buf, 8, 2);      // view: offset 8, 2 elements
a.buffer; a.byteOffset; a.byteLength; a.length; a.BYTES_PER_ELEMENT;
a.set([1,2], 0);
a.subarray(1, 3);      // VIEW, shares memory
a.slice(1, 3);         // COPY
```

TypedArrays support most array methods (`map`, `filter`, `reduce`, `sort`, `find`, `at`, `includes`) but **not** `push`, `pop`, `splice`, `concat` — the length is fixed. Methods that return a new array return a TypedArray of the same type.

Out-of-range writes wrap (or clamp for `Uint8ClampedArray`); out-of-bounds reads give `undefined`, and writes are silently ignored.

27.3 DataView

Explicit control over byte offset and endianness — the right tool for parsing binary formats.

```
const dv = new DataView(buf);
dv.setInt32(0, 42, /* littleEndian */ true);
dv.getInt32(0, true);
dv.getFloat64(8);       // defaults to BIG-endian
dv.getUint8(0); dv.setBigInt64(0, 1n);
```

TypedArrays use the **platform's** endianness (little-endian in practice); DataView lets you specify it, which is what network protocols and file formats require.

27.4 Base64 and Text Conversion

```
new TextEncoder().encode("héllo");            // Uint8Array (UTF-8)
new TextDecoder("utf-8").decode(bytes);       // string

// ES2025 base64 on Uint8Array:
bytes.toBase64(); Uint8Array.fromBase64(str);
bytes.toHex();    Uint8Array.fromHex(str);

// Legacy browser route:
btoa(binaryString); atob(base64);   // ⚠ latin1 only, breaks on non-ASCII
```

27.5 Atomics

```
Atomics.add(sharedInt32Array, index, value);
Atomics.load / store / exchange / compareExchange / sub / and / or / xor
Atomics.wait(arr, i, expected, timeout);     // blocks — workers only
Atomics.notify(arr, i, count);
Atomics.waitAsync(...)                        // non-blocking variant
```

CHAPTER 28

Intl — Internationalization

```
// Numbers, currency, units, percentages
new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR"
}).format(1234.5);
new Intl.NumberFormat("en", { notation: "compact" }).format(1234567);       // "1.2M"
new Intl.NumberFormat("en", { style: "unit", unit: "kilometer-per-hour"
}).format(50);
new Intl.NumberFormat("en", { maximumFractionDigits: 2 }).format(x);

// Dates and times
new Intl.DateTimeFormat("en-GB", { dateStyle: "full", timeZone: "Asia/Kolkata"
}).format(d);
new Intl.DateTimeFormat("en", { hour: "numeric", minute: "2-digit"
}).formatRange(a, b);

// Relative time
new Intl.RelativeTimeFormat("en", { numeric: "auto" }).format(-3, "hour");  // "3
hours ago"

// Lists
new Intl.ListFormat("en", { type:
"conjunction" }).format(["a","b","c"]);   // "a, b, and c"

// Plural categories
new Intl.PluralRules("en").select(1);     // "one"
new Intl.PluralRules("en").select(2);     // "other"

// Collation (sorting)
new Intl.Collator("de", { numericSort: true, sensitivity: "base" }).compare(a, b);
arr.sort(new Intl.Collator("en", { numeric: true }).compare);   // "item2" < "item10"

// Segmentation (graphemes, words, sentences)
[...new Intl.Segmenter("en", { granularity: "word" }).segment(text)];

// Display names
new Intl.DisplayNames(["en"], { type: "region" }).of("IN");     // "India"

// Duration (ES2025)
new Intl.DurationFormat("en", { style: "long" }).format({ hours: 1, minutes: 30
});

// Environment info
```

```
Intl.DateTimeFormat().resolvedOptions().timeZone;   // user's IANA timezone
Intl.supportedValuesOf("timeZone");
```

Reusing a formatter instance is significantly faster than calling `toLocaleString` in a loop.

PART V

Errors, Async, Modules

29	Errors and Exception Handling	100
30	The Event Loop and Concurrency Model	103
31	Promises	106
32	async / await	110
33	Async Iteration	113
34	Modules	115

CHAPTER 29

Errors and Exception Handling

IN THIS CHAPTER

29.1 The Error Hierarchy 100
29.2 Error Properties 100
29.3 Custom Errors 100
29.4 Throwing and Catching 101
29.5 Error Chaining 101
29.6 Async Errors 102
29.7 Global Handlers 102
29.8 Errors vs. Result Values 102

29.1 The Error Hierarchy

```
Error
├── EvalError          (legacy, not thrown by modern engines)
├── RangeError         value outside the allowed range
├── ReferenceError     undeclared identifier / TDZ access
├── SyntaxError        invalid source (also JSON.parse)
├── TypeError          wrong type / calling a non-function / invariant violation
├── URIError           malformed URI in encodeURI/decodeURI
└── AggregateError     multiple errors (Promise.any)
```

29.2 Error Properties

```
const e = new Error("something failed", { cause: originalError });   // cause: ES2022
e.message;    // "something failed"
e.name;       // "Error"
e.stack;      // non-standard but universal
e.cause;      // originalError
e.toString(); // "Error: something failed"
```

29.3 Custom Errors

```
class AppError extends Error {
  constructor(message, options) {
    super(message, options);
    this.name = this.constructor.name;
    Error.captureStackTrace?.(this, this.constructor);   // V8: trim the ctor frame
  }
}
```

```
class HttpError extends AppError {
  constructor(status, message, options) {
    super(message, options);
    this.status = status;
  }
  get isRetryable() { return this.status >= 500; }
}

throw new HttpError(503, "Service unavailable", { cause: netErr });
```

Always set `name` — `instanceof` fails across realms and after some bundler transforms, so a `name` or `code` field is a useful secondary check.

29.4 Throwing and Catching

```
throw new Error("msg");       // ✅
throw "msg";                  // ❌ legal but loses the stack trace

try { risky(); }
catch (e) {                   // binding optional since ES2019
  if (e instanceof HttpError) handleHttp(e);
  else if (e instanceof TypeError) handleType(e);
  else throw e;               // rethrow what you can't handle
}
finally { cleanup(); }
```

Never write an empty `catch {}`. Either handle, log with context, or rethrow (optionally wrapped with `cause`).

29.5 Error Chaining

```
try { await db.query(sql); }
catch (err) {
  throw new AppError("Failed to load user profile", { cause: err });
}

// Walking the chain:
let e = err;
while (e) { console.error(e.message); e = e.cause; }
```

29.6 Async Errors

```
// Promise rejection is caught by .catch or try/catch with await
try { await fetchData(); } catch (e) {}
fetchData().catch(handle);

// ⚠ NOT caught — the callback runs after try/catch has exited:
try { setTimeout(() => { throw new Error("x"); }, 0); } catch (e) {}

// ⚠ Missing await means the rejection escapes:
async function f() { doAsync(); }        // unhandled rejection
async function f() { return doAsync(); } // rejection propagates to the caller
```

29.7 Global Handlers

```
// Browser
window.addEventListener("error", e => report(e.error));
window.addEventListener("unhandledrejection", e => { report(e.reason);
e.preventDefault(); });

// Node
process.on("uncaughtException", (err, origin) => { log(err); process.exit(1); });
process.on("unhandledRejection", reason => { log(reason); });
```

Unhandled rejections terminate the process by default in Node ≥15. Global handlers are for logging and graceful shutdown, not for continuing as if nothing happened.

29.8 Errors vs. Result Values

Exceptions suit *exceptional* conditions. For expected failures (validation, parsing user input), returning a result is often clearer:

```
function parse(json) {
  try { return { ok: true, value: JSON.parse(json) }; }
  catch (e) { return { ok: false, error: e }; }
}
```

CHAPTER 30

The Event Loop and Concurrency Model

IN THIS CHAPTER

30.1 The Pieces 103
30.2 The Algorithm 103
30.3 Ordering Example 104
30.4 Timers 104
30.5 Node's Loop Phases 105
30.6 Blocking the Loop 105
30.7 True Parallelism 105

JavaScript has **one thread of execution** and never blocks on I/O. Concurrency comes from an event loop scheduling callbacks.

30.1 The Pieces

- **Call stack** — frames for currently executing functions.
- **Heap** — object memory.
- **Web APIs / libuv** — timers, network, file I/O; these run *outside* the JS thread.
- **Macrotask queue (task queue)** — timers, I/O, UI events, `setImmediate`.
- **Microtask queue** — promise reactions, `queueMicrotask`, `MutationObserver`, `process.nextTick` (own, higher-priority queue in Node).
- **Event loop** — the coordinator.

30.2 The Algorithm

```
loop forever:
  1. run one macrotask to completion (the call stack empties)
  2. drain the ENTIRE microtask queue (including microtasks queued during this
drain)
  3. (browser) run requestAnimationFrame callbacks, then style/layout/paint if due
  4. repeat
```

Key consequence: **microtasks always run before the next macrotask**, and an infinite chain of microtasks starves the loop entirely.

30.3 Ordering Example

```
console.log("1");

setTimeout(() => console.log("2"), 0);            // macrotask

Promise.resolve().then(() => console.log("3"));   // microtask

queueMicrotask(() => console.log("4"));           // microtask

(async () => {
  console.log("5");
  await null;
  console.log("6");                                // microtask after await
})();

console.log("7");

// Output: 1, 5, 7, 3, 4, 6, 2
```

Walk-through: synchronous code prints 1, 5, 7. Then microtasks drain in FIFO order: 3, 4, 6. Then the macrotask: 2.

30.4 Timers

```
setTimeout(fn, ms, ...args);      // returns an id (Node: a Timeout object)
clearTimeout(id);
setInterval(fn, ms);
clearInterval(id);
setImmediate(fn);                 // Node only — runs in the check phase
requestAnimationFrame(fn);        // browser — before the next paint (~16.7ms)
requestIdleCallback(fn);          // browser — when idle
queueMicrotask(fn);
```

`setTimeout(fn, 0)` does not run immediately — it runs after the current task and all microtasks, and browsers clamp nested timers to ≥4ms after 5 levels, and throttle timers in background tabs (often to ≥1000ms).

`setInterval` does not guarantee spacing; if the callback runs long, invocations bunch up. A self-rescheduling `setTimeout` is usually safer:

```
(function tick() { work(); setTimeout(tick, 1000); })();
```

30.5 Node's Loop Phases

```
timers → pending callbacks → idle/prepare → poll → check (setImmediate) → close
callbacks
```

`process.nextTick` callbacks run before promise microtasks, after each phase. Overusing it can starve I/O.

30.6 Blocking the Loop

Any long synchronous computation freezes everything — no rendering, no events, no timers.

```
// Yield periodically:
async function processAll(items) {
  for (let i = 0; i < items.length; i++) {
    process(items[i]);
    if (i % 100 === 0) await new Promise(r => setTimeout(r, 0));
  }
}

// Or move work off-thread:
const worker = new Worker("worker.js");            // browser
const { Worker } = require("node:worker_threads"); // Node
```

30.7 True Parallelism

JavaScript itself is single-threaded, but the platform offers parallelism: Web Workers, Service Workers, Worklets, Node `worker_threads`, `child_process`, and `SharedArrayBuffer` + `Atomics` for shared memory. Messages between threads are structured-cloned unless you transfer an `ArrayBuffer` or use shared memory.

CHAPTER 31

Promises

IN THIS CHAPTER

31.1	States	106	31.4	Combinators	107
31.2	Creating Promises	106	31.5	Common Mistakes	108
31.3	Consuming	107	31.6	Thenables	109

A Promise is an object representing the eventual result of an asynchronous operation.

31.1 States

- **pending** → **fulfilled** (with a value) or **rejected** (with a reason)
- Settling is **one-way and permanent.**

31.2 Creating Promises

```
const p = new Promise((resolve, reject) => {
  if (ok) resolve(value);
  else reject(new Error("failed"));
});

Promise.resolve(v);        // already fulfilled; passes thenables/promises through
Promise.reject(e);         // already rejected
Promise.try(fn);           // ES2025 — runs fn, capturing sync throws as rejections
Promise.withResolvers();   // ES2024 → { promise, resolve, reject }
```

```
// Promisifying a callback API:
const delay = ms => new Promise(r => setTimeout(r, ms));
const readFile = path => new Promise((res, rej) =>
  fs.readFile(path, (err, data) => err ? rej(err) : res(data))
);
```

The executor runs **synchronously.** Throwing inside it rejects the promise.

31.3 Consuming

```
p.then(onFulfilled, onRejected)
p.catch(onRejected)                // ≡ .then(undefined, onRejected)
p.finally(onFinally)               // ES2018 — no args, passes value/reason through
```

Every `then`/`catch`/`finally` returns a **new** promise, which is what makes chaining work:

```
fetch(url)
  .then(res => {
    if (!res.ok) throw new HttpError(res.status);   // throw → rejects the chain
    return res.json();                              // return a value → fulfills
  })
  .then(data => transform(data))
  .catch(err => fallback(err))       // catches ANY error above it
  .finally(() => hideSpinner());
```

Rules of chaining:

- Returning a **value** fulfills the next promise with it.
- Returning a **promise** adopts its eventual state (this is how flattening works).
- **Throwing** rejects the next promise.
- A `catch` that returns normally *recovers* the chain — subsequent `then`s run.

31.4 Combinators

```
Promise.all([p1, p2])          // all fulfilled → array of values (order preserved)
                               // ANY rejects → rejects immediately with that reason
Promise.allSettled([p1, p2])   // ES2020 — never rejects
                               // → [{status:"fulfilled",value},
{status:"rejected",reason}]
Promise.race([p1, p2])         // first to SETTLE (fulfill or reject) wins
Promise.any([p1, p2])          // ES2021 — first to FULFILL wins
                               // all reject → AggregateError with .errors
```

```
// Timeout pattern
const withTimeout = (p, ms) => Promise.race([
  p,
  new Promise((_, rej) => setTimeout(() => rej(new Error("timeout")), ms)),
]);

// Concurrency limit
async function pool(tasks, limit) {
  const results = [];
  const running = new Set();
  for (const task of tasks) {
    const p = task().finally(() => running.delete(p));
    running.add(p);
```

```
    results.push(p);
    if (running.size >= limit) await Promise.race(running);
  }
  return Promise.all(results);
}

// Retry with exponential backoff
async function retry(fn, attempts = 3, base = 200) {
  for (let i = 0; ; i++) {
    try { return await fn(); }
    catch (e) {
      if (i >= attempts - 1) throw e;
      await new Promise(r => setTimeout(r, base * 2 ** i));
    }
  }
}
```

Important: `Promise.all` starts **all** promises immediately — the array elements are already-running promises, not lazy tasks. To limit concurrency you must not create them all upfront.

31.5 Common Mistakes

```
// ❌ Forgetting to return — the chain doesn't wait
p.then(() => { doAsync(); }).then(() => console.log("too early"));
// ✅
p.then(() => doAsync()).then(() => console.log("correct"));

// ❌ The explicit-construction antipattern
new Promise(res => getData().then(res));
// ✅
getData();

// ❌ Nesting instead of chaining
a().then(x => b(x).then(y => c(y)));
// ✅
a().then(b).then(c);

// ❌ Unhandled rejection — nothing consumes the error
somePromise.then(handleSuccess);
// ✅
somePromise.then(handleSuccess).catch(handleError);

// ❌ Swallowing
p.catch(() => {});
```

31.6 Thenables

Any object with a callable `.then` is treated as a promise by `await` and `Promise.resolve`. This is how interop with old libraries works — and a subtle hazard if an object accidentally has a `then` property.

CHAPTER 32

async / await

IN THIS CHAPTER

32.1 Basics 110
32.2 Error Handling 111
32.3 Sequential vs Parallel 111
32.4 Loops 111
32.5 Top-Level `await` 112
32.6 Cancellation with AbortController 112

Syntactic sugar over promises, built on the generator machinery.

32.1 Basics

```
async function f() { return 1; }           // always returns a Promise
f();                                        // Promise { 1 }

async function g() { throw new Error("x"); }  // → rejected promise

const h = async () => { };
const obj = { async method() {} };
class C { async method() {} static async s() {} }
```

`await` unwraps a promise (or any thenable) and pauses the function; everything after it becomes a microtask continuation. `await` on a non-promise still yields to the microtask queue.

32.2 Error Handling

```
async function load() {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new HttpError(res.status);
    return await res.json();        // NOTE the await — see below
  } catch (err) {
    log(err);
    throw err;
  } finally {
    cleanup();
  }
}
```

`return await p` vs `return p` inside `try`: without `await`, the promise is returned before it settles, so a rejection escapes the local `catch`. Inside a `try` block, always `return await`.

32.3 Sequential vs Parallel

```
// ❌  Sequential — 3 seconds total
const a = await slow();
const b = await slow();
const c = await slow();

// ✅  Parallel — 1 second total
const [a, b, c] = await Promise.all([slow(), slow(), slow()]);

// ⚠  Starting early, awaiting later — beware unhandled rejections between the
two
const pa = slow(), pb = slow();
const a = await pa, b = await pb;
```

32.4 Loops

```
// Sequential (each waits for the previous)
for (const url of urls) { await fetch(url); }

// Parallel
await Promise.all(urls.map(url => fetch(url)));

// ❌  forEach does NOT await — the callback's promise is discarded
urls.forEach(async url => { await fetch(url); });  // finishes "instantly"

// Sequential reduce
await urls.reduce((p, url) => p.then(() => fetch(url)), Promise.resolve());
```

32.5 Top-Level `await`

Legal at the top level of an **ES module** only (ES2022) — not in scripts, not in CommonJS.

```
// module.mjs
const config = await fetch("/config.json").then(r => r.json());
export default config;
```

It delays evaluation of every module that imports this one. Use it for setup, not for anything slow or optional.

32.6 Cancellation with AbortController

Promises are not cancellable. The platform convention is `AbortSignal`:

```
const ac = new AbortController();
fetch(url, { signal: ac.signal }).catch(e => {
  if (e.name === "AbortError") return;   // expected
  throw e;
});
ac.abort();

AbortSignal.timeout(5000);           // auto-aborting signal
AbortSignal.any([s1, s2]);
signal.throwIfAborted();
signal.addEventListener("abort", cleanup);
```

Supporting it in your own async function means checking `signal.aborted` at await boundaries and listening for the `abort` event.

CHAPTER 33

Async Iteration

IN THIS CHAPTER

33.1 `for await...of` 113
33.2 Custom Async Iterables 113
33.3 Real-World Sources 114

33.1 `for await...of`

```
for await (const chunk of stream) { process(chunk); }
```

Works on async iterables (`Symbol.asyncIterator`), and also on sync iterables of promises — awaiting each in turn.

```
for await (const v of [Promise.resolve(1), Promise.resolve(2)]) {}   // 1, 2
```

Must appear inside an `async` function or at module top level. It is **sequential** by design; for parallelism use `Promise.all`.

33.2 Custom Async Iterables

```
const ticker = {
  async *[Symbol.asyncIterator]() {
    while (true) {
      await new Promise(r => setTimeout(r, 1000));
      yield Date.now();
    }
  },
};
```

33.3 Real-World Sources

```
// Web Streams
for await (const chunk of response.body) {}

// Node streams (async-iterable since v10)
for await (const line of readline.createInterface({ input })) {}

// Paginated APIs — see §17.6
```

Cleanup: `break` or `throw` inside `for await` invokes the iterator's `return()`, so `finally` blocks in async generators do run.

CHAPTER 34

Modules

IN THIS CHAPTER

34.1 ES Modules (ESM) — the Standard 115
34.2 Semantics 115
34.3 Loading Order 116
34.4 Dynamic Import 116
34.5 CommonJS (Node's Legacy System) 117
34.6 Configuring Module Type in Node 117
34.7 Browser Usage 118
34.8 Historical Formats 118

34.1 ES Modules (ESM) — the Standard

Exporting

```
export const a = 1;
export function f() {}
export class C {}
export { a, b as renamed };
export default something;              // one per module
export * from "./other.js";            // re-export all (not default)
export * as ns from "./other.js";      // ES2020
export { default } from "./other.js";
export { default as Thing } from "./other.js";
```

Importing

```
import defaultExport from "./mod.js";
import { a, b as renamed } from "./mod.js";
import defaultExport, { a } from "./mod.js";
import * as ns from "./mod.js";
import "./side-effect.js";                    // run only
const mod = await import("./mod.js");         // dynamic, returns a Promise
import data from "./data.json" with { type: "json" };   // import attributes, ES2025
import.meta.url;                              // module's own URL
import.meta.resolve("./x.js");
```

34.2 Semantics

- **Static structure:** imports/exports are hoisted and analyzed before execution, which enables tree-shaking and circular-dependency handling.
- **Live bindings:** importers see the current value of an exported binding, not a copy.
- **Singleton:** each module is evaluated once per realm and cached.
- **Always strict mode**, and top-level `this` is `undefined`.
- **Own scope:** top-level declarations are not global.
- **Deferred, async execution:** `<script type="module">` behaves like `defer`.
- **CORS applies** to cross-origin module fetches.
- Specifiers must be resolvable — browsers require a full path or an import map.

```
// counter.js
export let count = 0;
export const inc = () => count++;

// main.js
import { count, inc } from "./counter.js";
inc();
console.log(count);   // 1 — live binding
// count = 5;         ← TypeError: imported bindings are read-only
```

34.3 Loading Order

1. **Construction** — fetch and parse all modules, build the graph.
2. **Instantiation** — allocate exports, link imports (bindings exist but are in TDZ).
3. **Evaluation** — run module bodies depth-first, post-order.

Circular imports don't error, but a module may observe an uninitialized binding if it *uses* an import at evaluation time rather than inside a function.

34.4 Dynamic Import

```
// Code splitting / lazy loading
button.onclick = async () => {
  const { Chart } = await import("./chart.js");
  new Chart();
};

// Conditional / computed specifier
const locale = await import(`./i18n/${lang}.js`);
```

Works in scripts and CommonJS too, and is the standard way to load ESM from CJS.

34.5 CommonJS (Node's Legacy System)

```
const fs = require("node:fs");
const { readFile } = require("node:fs");

module.exports = fn;
module.exports.helper = fn;
exports.helper = fn;              // ⚠ reassigning `exports` breaks the link
```

Differences from ESM:

	ESM	CommonJS
Syntax	`import` / `export`	`require` / `module.exports`
Loading	async, static	sync, dynamic
Bindings	live	copied values
Analysis	static (tree-shakeable)	runtime
Strict mode	always	opt-in
Top-level `this`	`undefined`	`module.exports`
`__dirname` / `__filename`	no (use `import.meta.url`)	yes
Top-level await	yes	no
Circular deps	handled via TDZ	partial exports object

```
// ESM equivalents of __dirname:
import { fileURLToPath } from "node:url";
import { dirname } from "node:path";
const __dirname = dirname(fileURLToPath(import.meta.url));
// or Node ≥20.11:
import.meta.dirname;
```

34.6 Configuring Module Type in Node

```
// package.json
{
  "type": "module",        // .js files are ESM; use .cjs for CommonJS
  "exports": {
    ".": { "import": "./dist/index.mjs", "require": "./dist/index.cjs" },
    "./feature": "./dist/feature.js"
  },
  "imports": { "#internal/*": "./src/*" }
}
```

`.mjs` is always ESM; `.cjs` is always CommonJS. `exports` also *encapsulates* the package — paths not listed cannot be imported.

34.7 Browser Usage

```
<script type="module" src="./main.js"></script>
<script type="module">import { x } from "./m.js";</script>
<script nomodule src="./legacy.js"></script>
<link rel="modulepreload" href="./m.js">

<script type="importmap">
{ "imports": { "lodash": "https://cdn.example/lodash-es/lodash.js" } }
</script>
```

34.8 Historical Formats

- **IIFE / global namespace** — pre-2010, `window.MyLib = ...`
- **AMD** (`define`) — RequireJS, async browser loading
- **UMD** — a wrapper detecting AMD/CJS/global
- **CommonJS** — Node
- **ESM** — the standard; everything else is legacy or a build target

PART VI

Advanced and Environmental

35	Strict Mode	120
36	Memory Management and Garbage Collection	122
37	Metaprogramming Recap	124
38	Feature History by ECMAScript Version	125
39	Gotchas, Pitfalls, and Quirks	128
40	The Browser Environment	131
41	The Node.js Environment	134
42	Tooling Ecosystem	136
43	Appendices	138

CHAPTER 35

Strict Mode

IN THIS CHAPTER

35.1 Enabling 120
35.2 What Strict Mode Changes 120
35.3 Why It Matters 121

35.1 Enabling

```
"use strict";             // top of a file (whole script)

function f() {
  "use strict";           // top of a function body
}
```

Always strict, no directive needed: ES modules, class bodies (including all methods and static blocks), and anything inside them.

The directive must be the first statement (only comments may precede it) and must be a plain string literal — `'use strict'` after any code is ignored silently.

35.2 What Strict Mode Changes

Errors instead of silence:

```
"use strict";
undeclared = 1;                 // ReferenceError (was: implicit global)
Object.freeze(o); o.x = 1;      // TypeError (was: silent no-op)
delete Object.prototype;        // TypeError (was: false)
const o = { a: 1, a: 2 };       // allowed since ES6, was SyntaxError in ES5
function f(a, a) {}             // SyntaxError — duplicate params
0644;                           // SyntaxError — legacy octal literals banned
"\012";                         // SyntaxError — octal escapes banned
```

`this`
behavior:

```
function f() { return this; }
f();     // undefined in strict, globalThis in sloppy
// This also means primitives aren't boxed: (5).method() sees `this === 5`
```

Removed features:

- `with` — SyntaxError.
- `arguments.callee` and `arguments.caller` — TypeError on access.
- `fn.caller` and `fn.arguments` — TypeError.
- `arguments` no longer aliases named parameters.
- `eval` cannot introduce bindings into the surrounding scope.
- `eval` and `arguments` cannot be assigned to or used as binding names.

Reserved words: `implements`, `interface`, `let`, `package`, `private`, `protected`, `public`, `static`, `yield`.

Function declarations in blocks are block-scoped rather than hoisted to the function scope.

35.3 Why It Matters

Strict mode catches bugs, removes the slowest and least optimizable semantics, and reserves syntax for the future. Modern code is strict by default because everything is a module. Sloppy mode remains only for legacy scripts.

CHAPTER 36

Memory Management and Garbage Collection

IN THIS CHAPTER

36.1 Model 122
36.2 V8 Generational GC 122
36.3 Common Leak Sources 122
36.4 Diagnosing 123
36.5 Performance Practices 123

36.1 Model

Memory is allocated automatically on value creation and freed by the garbage collector when a value becomes **unreachable** from the roots (global object, current stack, closures in use).

Reference counting alone can't handle cycles, so engines use **mark-and-sweep**: mark everything reachable from the roots, sweep the rest.

36.2 V8 Generational GC

- **Young generation (nursery)** — small, collected frequently with a fast **Scavenger** (copying collector). Most objects die here.
- **Old generation** — objects surviving two scavenges get promoted; collected by **Mark-Compact**, with incremental marking, concurrent sweeping, and parallel compaction to limit pause times.
- Additional: **Orinoco** (concurrent/parallel GC), **major/minor GC**, pointer compression.

You cannot force GC from JavaScript (`global.gc()` only with `--expose-gc`).

36.3 Common Leak Sources

```
// 1. Forgotten timers/intervals
const id = setInterval(work, 1000);   // keeps its closure alive forever
clearInterval(id);

// 2. Detached DOM nodes held by JS
let cached = document.getElementById("x");
```

```
document.body.innerHTML = "";         // node removed from DOM, still referenced

// 3. Unremoved event listeners
el.addEventListener("click", handler);
el.removeEventListener("click", handler);   // needs the SAME reference
new AbortController();                       // or: { signal } then abort()

// 4. Unbounded caches / growing arrays
const cache = new Map();              // grows forever → use WeakMap or an LRU

// 5. Closures capturing more than needed
// 6. Accidental globals (sloppy mode)
// 7. Console.log'd objects in DevTools (retained while the console holds them)
```

36.4 Diagnosing

- Chrome DevTools **Memory** panel: heap snapshots, comparison view, allocation timeline, "Detached" filter.
- Node: `--inspect` + DevTools, `process.memoryUsage()`, `--max-old-space-size`, `heapdump`, clinic.js.
- Method: take a snapshot, exercise the suspect flow repeatedly, snapshot again, compare retained sizes and look for objects that should have been released.

36.5 Performance Practices

- Avoid changing an object's "shape" after creation — initialize all properties in the constructor so engines keep a stable hidden class.
- Keep arrays packed and monomorphic (all elements the same type).
- Reuse objects in hot loops instead of allocating; consider object pools for very hot paths.
- Prefer `Map` / `Set` over object/array scans for lookups.
- Don't micro-optimize before profiling — engines are extremely good at the common cases.

CHAPTER 37

Metaprogramming Recap

The tools for writing code that inspects or modifies program structure:

Tool	Purpose
`Proxy`	intercept fundamental object operations (§26.1 p. 92)
`Reflect`	perform those operations as functions (§26.2 p. 93)
`Symbol.*` well-known symbols	hook into language protocols (§25.3 p. 90)
Property descriptors	control writability, enumerability, configurability (§11 p. 46)
`Object.getOwnPropertyNames` / `Reflect.ownKeys`	introspect structure
`Function.prototype.toString`	read source text
`new Function` / `eval`	compile code at runtime
Getters/setters	computed and intercepted properties
`Symbol.toStringTag` / `hasInstance`	customize built-in behavior
Decorators (Stage 3)	declarative class transformations

37.1 `eval` and `new Function`

```
eval("var x = 1");           // DIRECT eval — can see and modify local scope
(0, eval)("var y = 1");      // INDIRECT eval — global scope only
new Function("a", "return a")(1);   // global scope, no closure access
```

Avoid both: they defeat optimization, break minifiers and bundlers, and open injection holes. They're also blocked by strict Content Security Policies. Legitimate uses are template engines and REPLs — and even those usually have safer alternatives.

CHAPTER 38

Feature History by ECMAScript Version

IN THIS CHAPTER

(1997) – ES3 (1999) 125
(2009) / ES5.1 (2011) 125
/ ES2015 — the big one 125
125
125
125
125
125
125
125
125
125
125
125
125
125
125
flight (Stage 3, likely next) 125

ES1 (1997) – ES3 (1999)

Core language: types, operators, functions, prototypes, `try/catch`, regular expressions, `do-while`, `switch`.

ES5 (2009) / ES5.1 (2011)

`"use strict"`; `JSON`; `Object.create`, `defineProperty`, `getOwnPropertyNames`, `keys`, `freeze`/`seal`/`preventExtensions`; `Array.isArray`; array iteration methods (`forEach`, `map`, `filter`, `reduce`, `reduceRight`, `some`, `every`, `indexOf`, `lastIndexOf`); `Function.prototype.bind`; `String.prototype.trim`; getters/setters in literals; trailing commas; reserved words as property names.

ES6 / ES2015 — the big one

`let`/`const`, arrow functions, classes, template literals, destructuring, default/rest/spread, modules, `Promise`, generators, iterators, `Symbol`, `Map`/`Set`/`WeakMap`/`WeakSet`, `Proxy`/`Reflect`, TypedArrays, `for...of`, computed property names, shorthand properties/methods, `Object.assign`, `Array.from`/`of`, `String.includes`/`startsWith`/`endsWith`/`repeat`, `Number.isInteger`/`isNaN`/`EPSILON`, `Math` additions, binary/octal literals, tail calls (spec only), block-scoped functions, `new.target`.

ES2016

`Array.prototype.includes`, exponentiation operator `**`.

ES2017

`async`/`await`, `Object.values`/`entries`/`getOwnPropertyDescriptors`, `String.padStart`/`padEnd`, trailing commas in function parameters, `SharedArrayBuffer`, `Atomics`.

ES2018

Async iteration (`for await...of`, async generators), object rest/spread, `Promise.prototype.finally`, RegExp: named capture groups, lookbehind, `s` (dotAll) flag, Unicode property escapes.

ES2019

`Array.flat`/`flatMap`, `Object.fromEntries`, `String.trimStart`/`trimEnd`, optional catch binding, `Symbol.description`, stable `Array.sort`, well-formed `JSON.stringify`, revised `Function.toString`, `JSON` superset (U+2028/2029 in strings).

ES2020

Optional chaining `?.`, nullish coalescing `??`, `BigInt`, dynamic `import()`, `Promise.allSettled`, `globalThis`, `String.matchAll`, `import.meta`, `export * as ns`, `for-in` order standardized.

ES2021

`String.replaceAll`, `Promise.any` + `AggregateError`, logical assignment (`&&=`, `||=`, `??=`), numeric separators `1_000`, `WeakRef`, `FinalizationRegistry`.

ES2022

Class fields (public/private, static), private methods, static initialization blocks, ergonomic brand checks (`#x in obj`), top-level `await`, `Object.hasOwn`, `Array.prototype.at` (and String/TypedArray), RegExp `d` flag (match indices), `Error.cause`.

ES2023

`findLast`/`findLastIndex`, change-by-copy methods (`toSorted`, `toReversed`, `toSpliced`, `with`), hashbang grammar, symbols as WeakMap keys.

ES2024

`Object.groupBy`, `Map.groupBy`, `Promise.withResolvers`, `ArrayBuffer.prototype.transfer` and resizable buffers, `Atomics.waitAsync`, RegExp `v` flag (unicodeSets), `String.isWellFormed`/`toWellFormed`.

ES2025

Iterator helpers, `Set` methods (union/intersection/difference/…), `RegExp.escape`, duplicate named capture groups, `Promise.try`, `Float16Array`, import attributes (`with { type: "json" }`), `JSON.parse` source access, `Math.sumPrecise`.

In flight (Stage 3, likely next)

Temporal, decorators, `using` / explicit resource management, records & tuples (paused), pattern matching (Stage 1), pipeline operator (Stage 2).

Track proposals at github.com/tc39/proposals. Stage 0 (idea) → 1 (proposal) → 2 (draft) → 2.7 → 3 (candidate, implementable) → 4 (finished, in the spec).

CHAPTER 39

Gotchas, Pitfalls, and Quirks

IN THIS CHAPTER

39.1 Equality and Coercion 128
39.2 this 128
39.3 Arrays 129
39.4 Async 129
39.5 Scope and Hoisting 129
39.6 Objects 129
39.7 Numbers and Strings 130
39.8 Regex 130
39.9 Things Best Avoided 130

39.1 Equality and Coercion

```
0.1 + 0.2 !== 0.3
NaN !== NaN
[] == false               // true
[] == ![]                 // true
null == undefined         // true, but null == 0 is false
"" == 0                   // true
typeof null === "object"
typeof NaN === "number"
0.5 + 0.1 == 0.6          // true (lucky) — never rely on this
9007199254740992 + 1 === 9007199254740992
```

39.2 this

```
const o = { name: "x", f: () => this.name };   // arrow ⇒  wrong `this`
const detached = o.method;                      // loses the receiver
setTimeout(o.method, 0);                        // same
arr.forEach(o.method);                          // same; pass a thisArg or bind
```

39.3 Arrays

```
[10, 9, 1].sort()                    // ["1","10","9"] ordering
new Array(3).map((_, i) => i)        // [<3 empty>] — map skips holes
["1","2","3"].map(parseInt)          // [1, NaN, NaN] — parseInt gets (val, index)
["1","2","3"].map(Number)            // [1, 2, 3] ✅
arr.forEach(...)                     // cannot break, cannot await
delete arr[0]                        // leaves a hole
typeof [] === "object"
[] + {} vs {} + []                   // depends on statement vs expression position
```

39.4 Async

```
// forEach doesn't await
items.forEach(async x => await save(x));

// Sequential when you meant parallel
for (const x of xs) await f(x);

// try/catch can't catch async callbacks
try { setTimeout(() => { throw e; }); } catch {}

// return vs return await inside try
// Promise.all fails fast — one rejection discards all results
// A rejected promise created but not awaited until later → unhandled rejection
warning
```

39.5 Scope and Hoisting

```
for (var i = 0; i < 3; i++) setTimeout(() => log(i));   // 3 3 3
typeof x;  let x;                    // ReferenceError (TDZ), not "undefined"
function f() { return \n {a:1}; }    // ASI returns undefined
```

39.6 Objects

```
{ a: 1 } === { a: 1 }                // false
Object.keys({ 2:0, 1:0, b:0, a:0 })  // ["1","2","b","a"] — integer keys first
JSON.stringify({ u: undefined })     // "{}"
Object.freeze(o)                     // shallow only
obj.constructor                      // spoofable, don't trust for type checks
"toString" in {}                     // true — inherited
```

39.7 Numbers and Strings

```
parseInt("08")           // 8 (fine now), but always pass a radix
parseInt(0.0000005)      // 5
(0.615).toFixed(2)       // "0.61"
"😀 ".length              // 2
"b" + "a" + +"a" + "a"   // "baNaNa"
Math.max()               // -Infinity
[].every(x => false)     // true — vacuous truth
```

39.8 Regex

```
const re = /a/g;
re.test("a"); re.test("a");   // true, false — lastIndex statefulness
str.match(/x/g)               // null, not [] — always guard
/(a+)+$/                      // catastrophic backtracking
```

39.9 Things Best Avoided

`with`, `eval`, `document.write`, `var`, `==` (except `== null`), `new Array/Object/String/Number/Boolean`, `arguments`, `for...in` on arrays, string-based `setTimeout("code")`, `__proto__` mutation, modifying built-in prototypes, `void 0` where `undefined` reads better, labels for anything but nested loop exits.

CHAPTER 40

The Browser Environment

IN THIS CHAPTER

40.1 Global Objects 131
40.2 The DOM 131
40.3 Events 132
40.4 Networking 132
40.5 Storage and Workers 132
40.6 Observers and Timing 132
40.7 Security Essentials 133

Not part of ECMAScript, but where most JavaScript runs. Brief orientation:

40.1 Global Objects

`window` (`globalThis`), `document`, `navigator`, `location`, `history`, `screen`, `console`, `localStorage`/`sessionStorage`, `crypto`, `performance`.

40.2 The DOM

```
document.querySelector(sel);  document.querySelectorAll(sel);   // static NodeList
document.getElementById(id);
document.createElement(tag);
el.append(...nodes);  el.prepend();  el.before();  el.after();  el.remove();
el.replaceChildren(...);
el.textContent;  el.innerHTML;   // innerHTML is an XSS vector — sanitize
el.classList.add/remove/toggle/contains/replace
el.dataset.key;                  // data-key attribute
el.getAttribute / setAttribute / hasAttribute / removeAttribute
el.closest(sel);  el.matches(sel);
el.getBoundingClientRect();
el.style.setProperty("--x", v);
```

40.3 Events

```
el.addEventListener("click", handler, { once, passive, capture, signal });
el.removeEventListener("click", handler);
el.dispatchEvent(new CustomEvent("my", { detail, bubbles: true }));

e.preventDefault();  e.stopPropagation();  e.stopImmediatePropagation();
e.target;  e.currentTarget;
```

Phases: capture (window → target), target, bubble (target → window). **Delegation** — one listener on a container using `e.target.closest(...)` — is the standard pattern for dynamic lists.

40.4 Networking

```
const res = await fetch(url, { method, headers, body, signal, credentials });
res.ok; res.status; await res.json(); await res.text(); await res.blob();

new WebSocket(url);
new EventSource(url);            // server-sent events
navigator.sendBeacon(url, data); // fire-and-forget on unload
```

40.5 Storage and Workers

```
localStorage.setItem(k, JSON.stringify(v));   // sync, ~5MB, strings only, origin-
scoped
sessionStorage                                 // per tab
indexedDB                                      // async, large, structured
document.cookie                                // sent with requests, small
caches                                         // Service Worker cache API

new Worker("w.js");  worker.postMessage(data);  worker.onmessage = e => e.data;
navigator.serviceWorker.register("/sw.js");
```

40.6 Observers and Timing

```
new IntersectionObserver(cb).observe(el);   // visibility
new MutationObserver(cb).observe(el, { childList: true, subtree: true });
new ResizeObserver(cb).observe(el);
new PerformanceObserver(cb).observe({ type: "largest-contentful-paint", buffered:
true });
performance.now();  performance.mark();  performance.measure();
```

40.7 Security Essentials

- **XSS** — never inject untrusted strings via `innerHTML`; prefer `textContent`, and use DOMPurify or Trusted Types when HTML is required.
- **CSP** — a `Content-Security-Policy` header restricts sources and blocks `eval`.
- **CORS** — cross-origin requests need server opt-in.
- **CSRF** — use `SameSite` cookies plus tokens.
- **Prototype pollution** — §12.8 p. 50.
- Never store secrets in client-side JavaScript.

CHAPTER 41

The Node.js Environment

IN THIS CHAPTER

41.1 Globals 134
41.2 Core Modules 134
41.3 Process and Environment 135
41.4 Streams and EventEmitter 135
41.5 Notes 135

41.1 Globals

`process`, `Buffer`, `__dirname`/`__filename` (CJS only), `require`/`module`/`exports` (CJS), `global`/`globalThis`, `console`, timers, `structuredClone`, `fetch` (v18+), `URL`, `TextEncoder`/`TextDecoder`, `AbortController`, `crypto`.

41.2 Core Modules

```
import fs from "node:fs/promises";     // promise-based file system
import path from "node:path";
import os from "node:os";
import url from "node:url";
import http from "node:http";
import crypto from "node:crypto";
import { EventEmitter } from "node:events";
import { Readable, pipeline } from "node:stream";
import { Worker } from "node:worker_threads";
import { spawn, execFile } from "node:child_process";
import { test } from "node:test";       // built-in test runner
import util from "node:util";
```

The `node:` prefix is preferred — it's unambiguous and cannot be shadowed by a package.

41.3 Process and Environment

```
process.env.NODE_ENV;
process.argv;               // [node, script, ...args]
process.exit(code);         // avoid — let the loop drain
process.cwd();
process.on("SIGINT", shutdown);
process.memoryUsage();
process.nextTick(fn);
```

41.4 Streams and EventEmitter

```
class Bus extends EventEmitter {}
bus.on("event", fn); bus.once(); bus.off(); bus.emit("event", data);
// Always handle "error" — an unhandled 'error' event throws.

await pipeline(createReadStream(src), createGzip(), createWriteStream(dst));
for await (const chunk of readable) {}
```

41.5 Notes

- CommonJS vs ESM: §34.5 p. 117–34.6.
- Buffers are `Uint8Array` subclasses with extra encoding methods.
- Blocking the loop blocks the whole server — offload CPU work to `worker_threads`.
- Cluster / PM2 / container replicas provide multi-core scaling.

CHAPTER 42

Tooling Ecosystem

Category	Tools
Runtimes	Node.js, Deno, Bun, browsers, workers/edge (Cloudflare, Vercel)
Package managers	npm, pnpm, yarn, bun
Bundlers	Vite (Rollup/esbuild), Rollup, esbuild, webpack, Parcel, Turbopack, Rspack
Transpilers	Babel, SWC, esbuild, TypeScript
Type systems	TypeScript, JSDoc + `checkJs`, Flow
Linters/Formatters	ESLint, Biome, oxlint, Prettier
Test runners	Vitest, Jest, `node:test`, Playwright, Cypress, Testing Library
Frameworks	React, Vue, Svelte, Angular, Solid, Next.js, Nuxt, SvelteKit, Astro, Remix
Server	Express, Fastify, Hono, NestJS, Koa
Monorepo	Turborepo, Nx, pnpm workspaces, Changesets
Debug/Profile	Chrome DevTools, `node --inspect`, clinic.js, Lighthouse

42.1 Semantic Versioning

`MAJOR.MINOR.PATCH` — breaking / feature / fix. Ranges: `^1.2.3` allows minor+patch, `~1.2.3` allows patch only, `1.2.3` is exact. Lockfiles pin the resolved tree; commit them.

42.2 Baseline Project Config

```
// package.json
{
  "name": "app",
  "type": "module",
  "engines": { "node": ">=20" },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest",
    "lint": "eslint .",
    "format": "prettier --write ."
```

```
    }
  }
```

CHAPTER 43

Appendices

IN THIS CHAPTER

A Truthiness Quick Card 138
B Choosing a Data Structure 138
C Copying Cheat Sheet 138
D Modern Style Defaults 139
E Debugging Toolkit 139
F Specification and Reference Links 139
G Learning Path from Here 140

A Truthiness Quick Card

Falsy (8): `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` **Notable truthy:** `[]`, `{}`, `"0"`, `"false"`, `" "`, `function() {}`, `Infinity`, `-1`, `new Boolean(false)`

B Choosing a Data Structure

Need	Use
Ordered list, index access	`Array`
Unique values, fast membership	`Set`
Any-type keys, insertion order, frequent add/delete	`Map`
Object-keyed metadata that shouldn't leak memory	`WeakMap`
Fixed-shape record, JSON round-trip	plain object
Binary data	TypedArray / `DataView`
Large integers	`BigInt`
Dictionary with untrusted keys	`Map` or `Object.create(null)`

C Copying Cheat Sheet

Goal	Code
Shallow copy object	`{...o}` / `Object.assign({}, o)`
Shallow copy array	`[...a]` / `a.slice()`
Deep copy	`structuredClone(o)`

Goal	Code
Copy with descriptors	`Object.create(Object.getPrototypeOf(o), Object.getOwnPropertyDescriptors(o))`
Merge	`{...a, ...b}`
Copy with one change	`{...o, k: v}` / `arr.with(i, v)`

D Modern Style Defaults

```
const / let, never var
=== over ==  (except `x == null`)
arrow functions for callbacks; method shorthand in objects/classes
template literals over concatenation
destructuring for parameters and returns
default parameters over `||` fallbacks
?? over || when 0 and "" are valid values
?. over long && chains
async/await over raw .then chains
for...of over for...in for arrays
map/filter/reduce for transformations; for...of when you need break/await
ES modules
named exports over default exports (better refactoring and tree-shaking)
throw Error subclasses, never strings
early return over deep nesting
```

E Debugging Toolkit

```
console.log / warn / error / info / debug
console.table(arrayOfObjects)
console.dir(obj, { depth: null })
console.group() / groupEnd()
console.time("x") / timeEnd("x")
console.count() / countReset()
console.trace()
console.assert(cond, msg)
debugger;

// Trace a value through a chain without breaking it:
const tap = x => (console.log(x), x);
```

Beyond logging: conditional breakpoints, logpoints, `Object.observe`-style Proxy logging (§26.1 p. 92), performance profiles, and source maps for debugging built code.

F Specification and Reference Links

- **ECMA-262 spec** — tc39.es/ecma262 (authoritative; `searchfox`-style search at tc39.es)

- **TC39 proposals** — github.com/tc39/proposals
- **MDN Web Docs** — developer.mozilla.org (the best practical reference)
- **Can I Use** — caniuse.com (browser support)
- **Node.js docs** — nodejs.org/api
- **V8 blog** — v8.dev/blog (engine internals and performance)

G Learning Path from Here

1. **Solidify fundamentals** — closures, `this`, prototypes, the event loop. These four explain most confusing behavior.
2. **Read the spec for one operator** — try the `+` operator or `==`. It demystifies coercion permanently.
3. **Add TypeScript** — it forces you to make the implicit types in your head explicit.
4. **Learn the platform** — DOM, fetch, streams, workers for browsers; fs, streams, http for Node.
5. **Study a real codebase** — read the source of a small library you use.
6. **Build with constraints** — no framework, no dependencies, then compare with the framework version.

— End of reference —

Section Index

Section Index

Every section title in alphabetical order. Chapter titles are marked "(chapter)".

A

Abstract Class Pattern, **54**
Abstract Operations, The, **15**
Accessing and Setting the Prototype, **48**
Accessing Properties, **42**
Algorithm, The, **103**
Appendices (chapter), **138**
arguments Object, The, **32**
Arithmetic, **21**
Arithmetic, **79**
Array Destructuring, **61**
Array-Likes, **60**
ArrayBuffer, **94**
Arrays, **129**
Arrays (chapter), **56**
Arrow Functions in Detail, **32**
Assignment Operators, **21**
Async, **129**
async / await (chapter), **110**
Async Errors, **102**
Async Generators, **68**
Async Iteration (chapter), **113**
Atomics, **96**

B

Base64 and Text Conversion, **95**
Baseline Project Config, **136**
Basics, **110**
BigInt, **9**
BigInt (chapter), **77**
Binding Rules, in Priority Order, The, **38**
Bitwise Operators, **22**
Blocking the Loop, **105**
Blocks and Empty Statements, **25**
Boolean, **7**
break, continue, and Labels, **27**
Browser Usage, **118**
Built-in Iterables, **65**

C

call, apply, bind, **34**
Cancellation with AbortController, **112**
Case Sensitivity, **2**
Catastrophic Backtracking (ReDoS), **83**
Character Set and Encoding, **2**
Checking for Properties, **43**
Choosing a Data Structure, **138**
Class Limitations, **54**
Classes (chapter), **51**
Classic Pitfalls, **37**
Closures (chapter), **36**
Combinators, **107**
Comma Operator, The, **24**
Comments, **3**
Common Array Recipes, **59**
Common Leak Sources, **122**
Common Mistakes, **108**
CommonJS (Node's Legacy System), **117**
Comparing Objects, **44**
Comparison and Sorting, **73**
Comparison Operators, **22**
Complete Method Reference, **71**
Conditional (Ternary) Operator, **23**
Conditionals, **25**
Configuring Module Type in Node, **117**
const Immutability Is Shallow, **11**
Constructor Function Inheritance (Pre-ES6), **49**
Consuming, **107**
Copying and Flattening, **59**
Copying and Merging, **44**
Copying Cheat Sheet, **138**
Core Modules, **134**
Creating, **81**
Creating and Using, **90**
Creating Arrays, **56**

Section Index

Creating Objects, **41**
Creating Promises, **106**
Creating Strings, **70**
Custom Async Iterables, **113**
Custom Errors, **100**

D

DataView, **95**
Date and Temporal (chapter), **78**
Date Object, The, **78**
debugger, **28**
Debugging Toolkit, **139**
Declaration and Expression, **51**
Declaration Forms, **11**
Declarations as Statements, **28**
Deep Clone Caveats, **86**
Defining Properties, **47**
Delegation with yield*, **67**
Descriptor Shapes, **46**
Destructuring in Parameters, **62**
Destructuring, Spread, and Rest (chapter), **61**
Detecting Types Reliably, **10**
Diagnosing, **123**
DOM, The, **131**
Dynamic Import, **116**

E

Enabling, **120**
Enumerating Properties, **43**
Equality and Coercion, **128**
Error Chaining, **101**
Error Handling, **111**
Error Hierarchy, The, **100**
Error Properties, **100**
Errors and Exception Handling (chapter), **100**
Errors vs. Result Values, **102**
ES Modules (ESM) — the Standard, **115**
ES1 (1997) – ES3 (1999), **125**
ES2016, **125**
ES2017, **125**
ES2018, **125**
ES2019, **125**
ES2020, **125**
ES2021, **125**
ES2022, **125**
ES2023, **125**
ES2024, **125**
ES2025, **125**
ES5 (2009) / ES5.1 (2011), **125**
ES6 / ES2015 — the big one, **125**
eval and new Function, **124**
Events, **132**
Extending Built-ins, **54**

F

Feature History by ECMAScript Version (chapter), **125**
Fields vs. Prototype Methods, **52**
Five Ways to Define a Function, The, **30**
Flags, **81**
Floating-Point Strategy, **75**
for await...of, **113**
Formatting, **79**
Full Anatomy, **51**
Full Built-in Chain, The, **50**
Function Properties and Methods, **35**
Functions (chapter), **30**

G

Generators (chapter), **66**
Getters and Setters, **78**
Getters and Setters (Accessor Properties), **44**
Global Handlers, **102**
Global Objects, **131**
Global Symbol Registry, The, **90**
Globals, **134**
Gotchas, Pitfalls, and Quirks (chapter), **128**
Grammar Notes, **5**
Grouping and Expression Statements, **24**

H

Higher-Order Functions, **34**
Historical Formats, **118**
Hoisting, **12**

I

Identifiers, **3**
Immediately Invoked Function Expressions (IIFE), **33**
Implicit Globals and globalThis, **13**
In flight (Stage 3, likely next), **125**
Inheritance with extends and super, **53**

Intl — Internationalization (chapter), 97
Iteration Method Signatures, 58
Iterator Helpers (ES2025), 65
Iterators and Iterables (chapter), 64

J

JSON (chapter), 85
JSON Format Rules, 86
JSON.parse, 86
JSON.stringify, 85

K

Known Date Problems, 79

L

lastIndex Statefulness, 83
Learning Path from Here, 140
length and Sparse Arrays, 56
Lexical Structure (chapter), 2
Literals, 4
Loading Order, 116
Logical Operators and Short-Circuiting, 23
Loops, 26
Loops, 111
Loops and Per-Iteration Bindings, 13
Loose Equality == — The Full Algorithm, 18
Losing this, 38

M

Manual Iteration and Early Exit, 65
Map, 87
Map, Set, WeakMap, WeakSet, WeakRef (chapter), 87
Match Object, The, 83
Math Reference, 75
Mechanism, The, 36
Memory Management and Garbage Collection (chapter), 122
Metaprogramming Recap (chapter), 124
Methods, 83
Mixins and Composition, 50
Model, 122
Modern Style Defaults, 139
Modules (chapter), 115
Mutating Methods, 57

N

Networking, 132
new.target, 35
Node's Loop Phases, 105
Non-Mutating Methods, 57
Notes, 135
Null, 7
Number, 8
Number and Math (chapter), 74
Number Instance Methods, 74
Number Statics, 74
Numbers and Strings, 130

O

Object, 9
Object Destructuring, 61
Object Literal Syntax, 41
Object Statics Reference, 44
Object.is — SameValue, 19
Objects, 129
Objects (chapter), 41
Observers and Timing, 132
Operator Precedence Table, 20
+ Operator, The, 17
Operators and Expressions (chapter), 20
Optional Chaining ?. (ES2020), 23
Ordering Example, 104

P

Parameters, 31
Pattern Syntax, 81
Performance Notes, 60
Performance Practices, 123
Pieces, The, 103
Practical Uses, 67
Private Members #, 53
Process and Environment, 135
Promises (chapter), 106
Property Descriptors and Object Meta-Operations (chapter), 46
Property Key Types and Ordering, 42
Prototype Chain, The, 48
Prototype Pollution (Security), 50
prototype vs [[Prototype]], 49
Prototypes and Inheritance (chapter), 48

Section Index

Proxy, **92**
Proxy and Reflect (chapter), **92**

R

Real-World Sources, **114**
Recursion, **33**
Reflect, **93**
Regex, **130**
Regular Expressions (chapter), **81**
Relational Comparison, **19**
replace in Depth, **72**
Reserved Words, **4**
Rest, **62**
Return Values, **32**
return() and throw(), **67**

S

Scope and Hoisting, **129**
Scope Chain, The, **12**
Scope Types, **12**
Sealing Levels, **47**
Security Essentials, **133**
Semantic Versioning, **136**
Semantics, **115**
Semicolons and Automatic Semicolon Insertion (ASI), **4**
Sequential vs Parallel, **111**
Set, **88**
Setting and Deleting, **42**
Shadowing, **14**
Sorting in Depth, **58**
Specification and Reference Links, **139**
Spread, **62**
Spread and Rest ..., **24**
Spread vs Rest Summary, **63**
Statements and Control Flow (chapter), **25**
States, **106**
Storage and Workers, **132**
Streams and EventEmitter, **135**
Strict Equality ===, **19**
Strict Mode (chapter), **120**
String, **8**
String (chapter), **70**
String Immutability and Performance, **73**
super, **39**
Symbol, **8**
Symbol (chapter), **90**
Symbols as Property Keys, **91**
Syntax and Basics, **66**

T

Tagged Template Literals, **24**
Template Literals, **71**
Temporal (Stage 3, shipping), **79**
Temporal Dead Zone (TDZ), The, **13**
The Browser Environment (chapter), **131**
The Event Loop and Concurrency Model (chapter), **103**
The Node.js Environment (chapter), **134**
Thenables, **109**
Things Best Avoided, **130**
this, **128**
this and Execution Context (chapter), **38**
this in Various Contexts, **39**
Throwing and Catching, **101**
Timers, **104**
ToBoolean, **15**
ToNumber, **16**
Tooling Ecosystem (chapter), **136**
Top-Level await, **112**
ToPrimitive and Hints, **17**
ToString, **17**
True Parallelism, **105**
Truthiness Quick Card, **138**
try / catch / finally / throw, **27**
Two Protocols, The, **64**
Two-Way Communication, **66**
Type Coercion and Conversion (chapter), **15**
Type System at a Glance, The, **6**
Typed Arrays, ArrayBuffer, DataView (chapter), **94**
TypedArray Types, The, **94**
typeof, void, delete, **24**
Types and Values (chapter), **6**

U

Undefined, **7**
Useful Patterns, **84**
Uses, **36**
UTF-16, Code Units, and Code Points, **70**

Section Index

V

V8 Generational GC, **122**
Variables, Scope, and Hoisting (chapter), **11**

W

WeakMap and WeakSet, **88**
WeakRef and FinalizationRegistry (ES2021), **89**
Well-Known Symbols, **90**
Well-Known Symbols as Hooks, **47**
What Consumes Iterables, **64**
What new Actually Does, **49**
What Strict Mode Changes, **120**
Whitespace and Line Terminators, **2**
Why It Matters, **121**
with (Deprecated), **27**
Wrapper Objects (Boxing), **9**